# MLOps Projects Overview

## Introduction

# MLOps Projects Overview and Pipelines

## Overview

### Agenda

- [08:15 – 09:00] – Lecture: MLOps vs DevOps
- [09:15 – 10:00] – Lecture: MLOps stages
- [10:15 – 11:00] – Follow-along exercises: Cookiecutter Data Science
- [11:15 – 12:00] – Follow-along exercises: Pipeline, CI/CD, and cloud storage

### Preparation

For the exercises:

- Cookiecutter Data Science: Install
- Cookiecutter Data Science: Opinions on structure
- Cookiecutter Data Science: Using the template
- Cookiecutter Data Science: Github repo
- boto3 docs: Using S3
- GitHub Actions: Understanding GitHub Actions
- MinIO docs: Start MinIO with Docker

For the lecture:

- ML-Ops.org: Motivation
- ML-Ops.org: MLOps principles
- ML-Ops.org: Three levels of ML Software
- ML-Ops.org: End-to-End ML
- Google & MLOps levels

- ML Engineering, Chapter 2 (http://bit.ly/MLEbook-Chapter2)

### Notes

- The exercises will be a follow-along coding session, mainly using terminal/CLI
- Make sure to install Cookiecutter Data Science and test that the `ccds` command works for you.
- The exercises build a whole project from scratch, so make sure you have a working Python environment, Docker (for MinIO), and a github.com account.

## Lecture Intro

# MLOps Projects Overview and Pipelines

## Outline

1. MLOps vs DevOps
2. MLOps principles
3. MLOps in organisations
4. Stages of maturity
5. ML software levels
    - Data processing
    - Model experimentation
    - Code deployment
6. Interactive session - open questions, sliders, and quizzes on all of the above





## Continuous Feedback Board

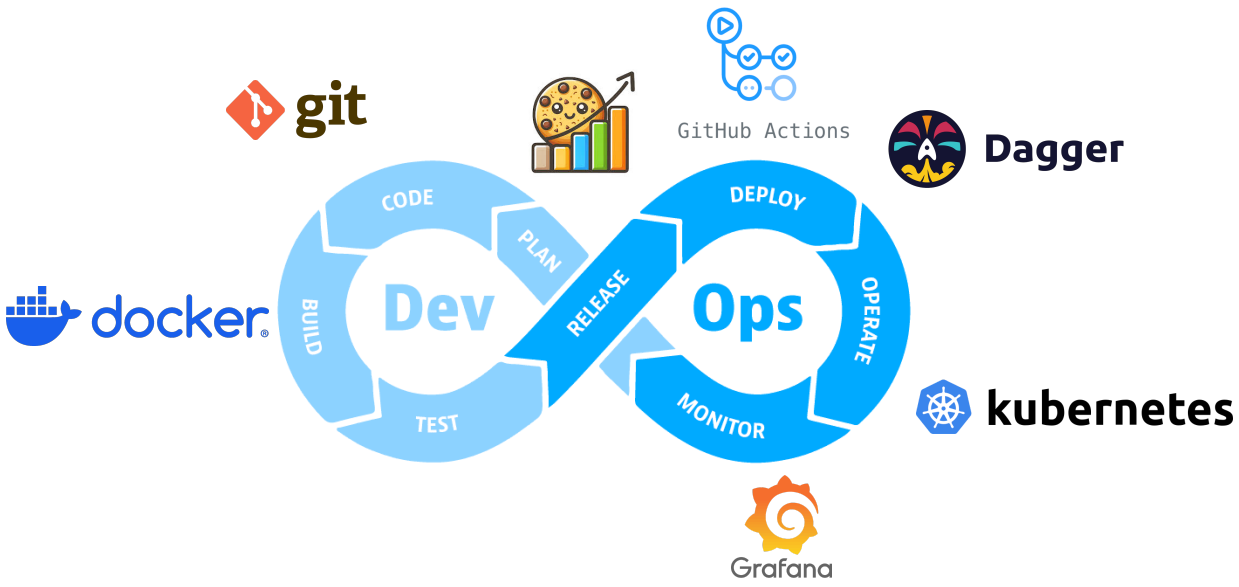
https://miro.com/app/board/uXjVJQwy4bs=/




## MLOps vs DevOps

# MLOps vs DevOps

## First: DevOps




Modified, from https://www.dynatrace.com/news/blog/what-is-devops/

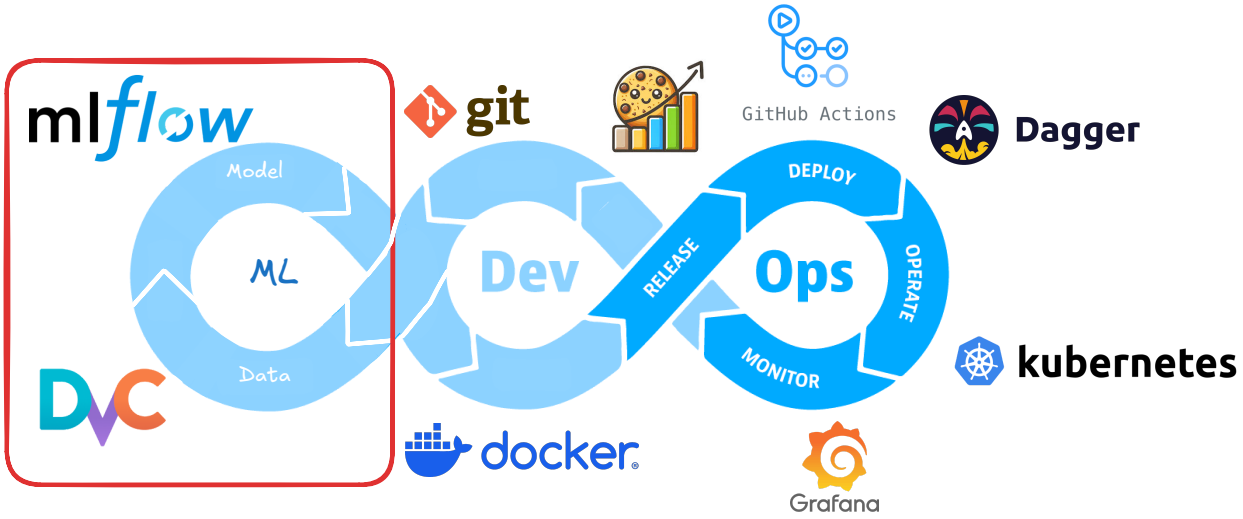
### Development

- **Plan**: Write down goals, tasks, and project rules, organize code with *CCDS*.
- **Code**: Write the code, version control with *Git*.
- **Build**: Package the program, build with *Docker*.
- **Test**: Check for bugs and errors, use unit/integration tests.

### Operations

- **Release**: Prepare software to go live.
- **Deploy**: Implement it in production, automate with *Github Actions* and *Dagger*.
- **Operate**: Keep servers running, organize containers with *Kubernetes*.
- **Monitor**: Watch system health, visualize and alert with *Grafana*.

## And MLOps?

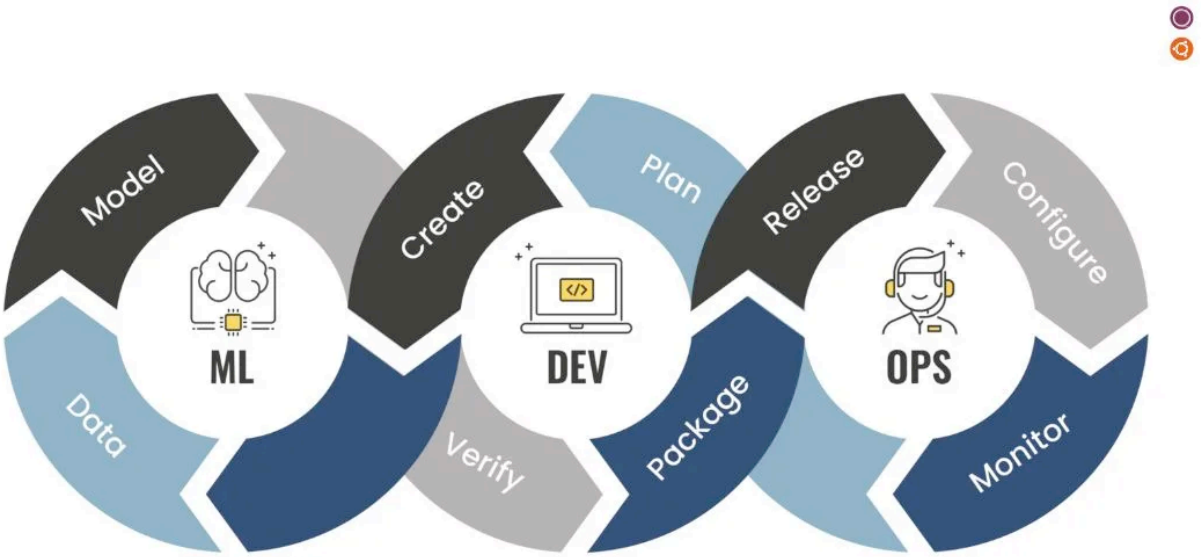



### ML

- **Data**: External info (vs internal code), version with *DVC*.
- **Model**: Machine learning model, track with *MLflow*.

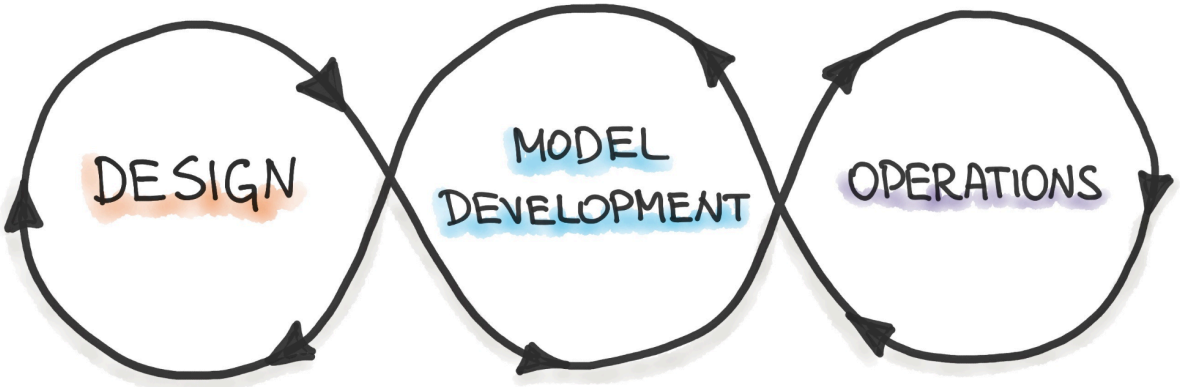
## Different graphics

### Slightly less ugly




From https://ubuntu.com/blog/what-is-mlops

# MLOps



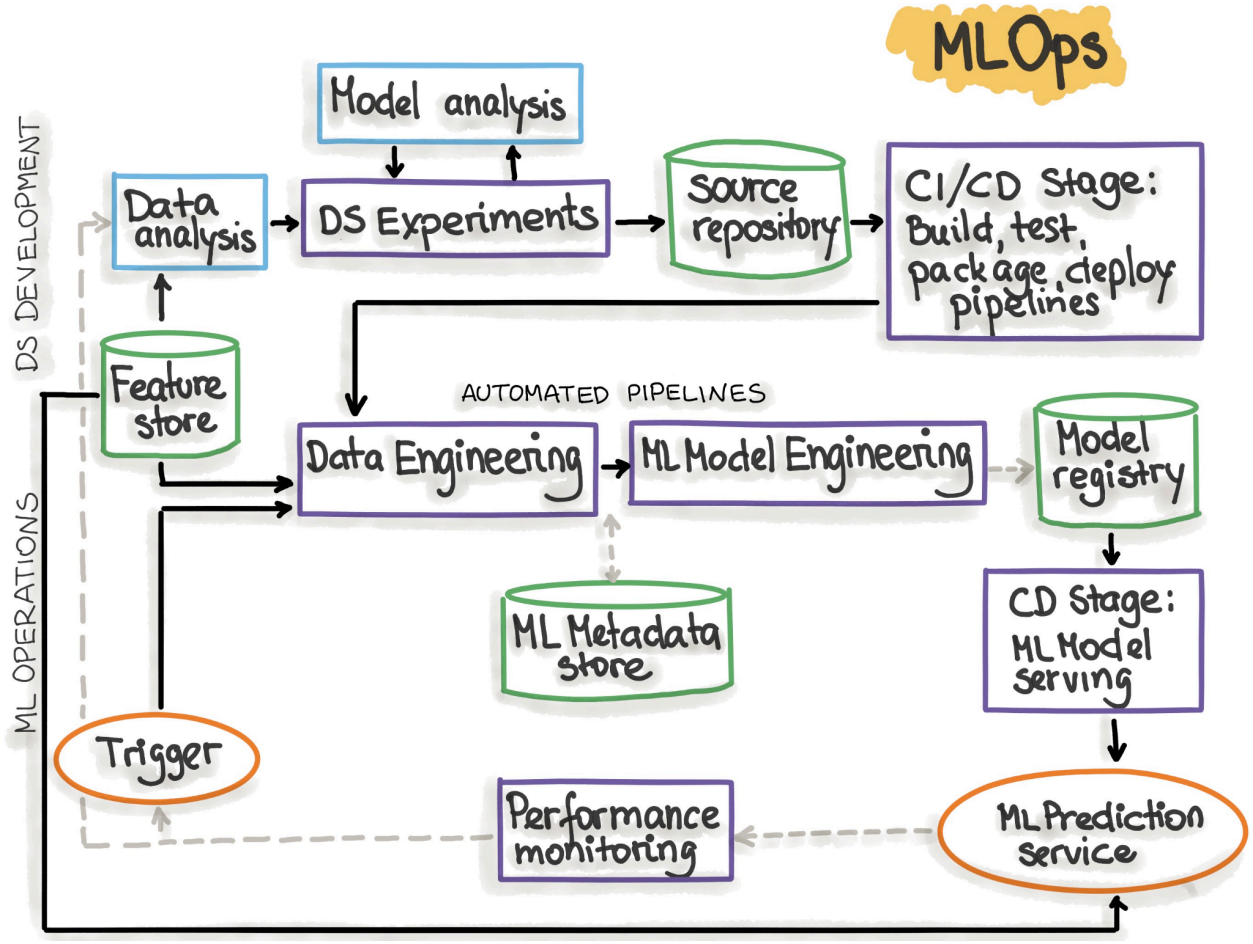

- Requirements Engineering
- ML Use-Cases Prioritization
- Data Availability Check

- Data Engineering
- ML Model Engineering
- Model Testing & Validation

- ML Model Deployment
- CI/CD Pipelines
- Monitoring & Triggering

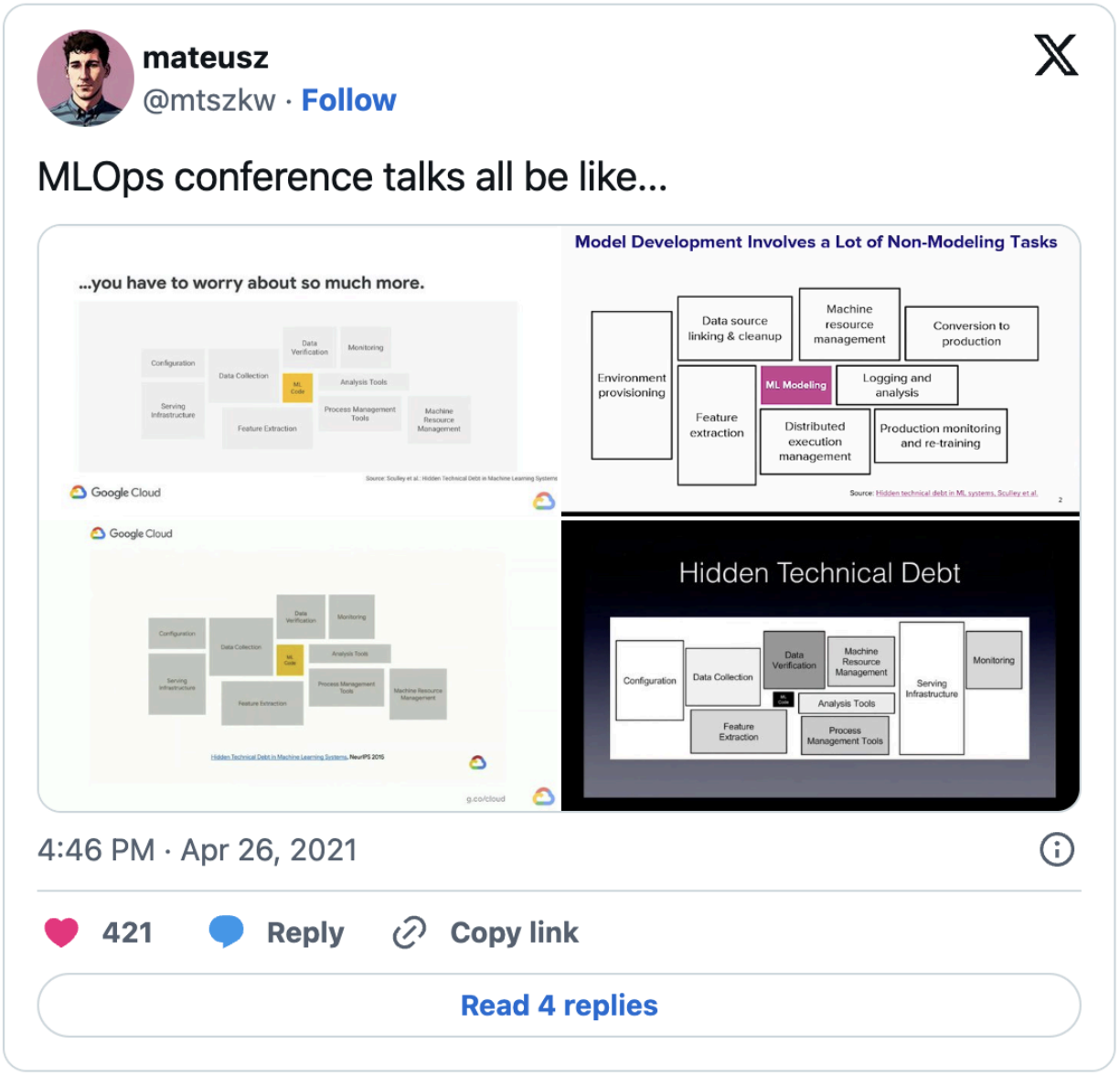
From https://ml-ops.org/content/mlops-principles

### Slightly less abstract

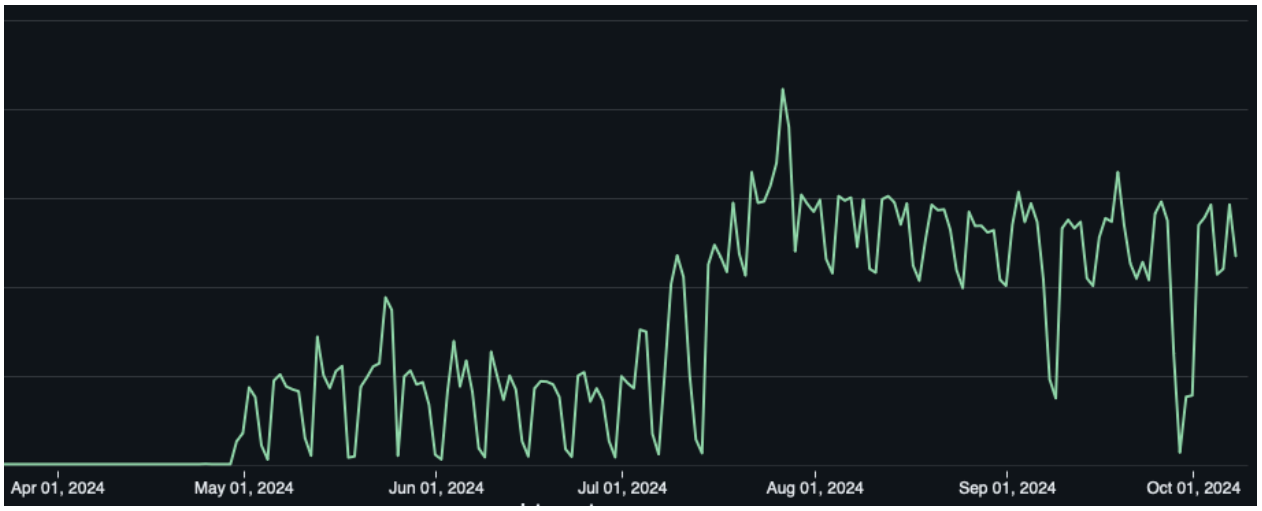



From https://ml-ops.org/content/mlops-principles

## Data and models add complexity




Source for original graphic: https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf




Page views over time

- Element of unpredictability
- Data + artifact versioning
- Changes over time

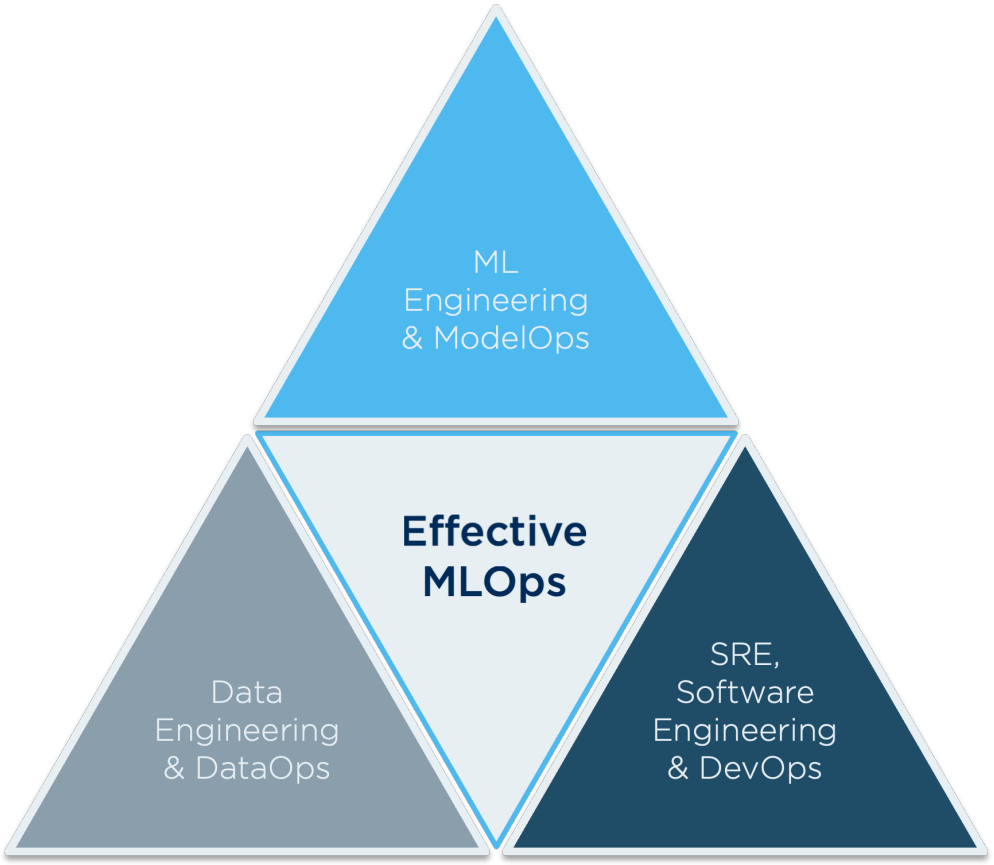
→ So much to keep track of

## MLOps Elements

# MLOps Elements

Based on https://ml-ops.org/content/mlops-principles

## Three Components of MLOps Pipelines




From https://ml-architects.ch/blog_posts/mlops_maturity_model.html

## Data

1. Extraction
2. Validation
3. Preparation

## Model

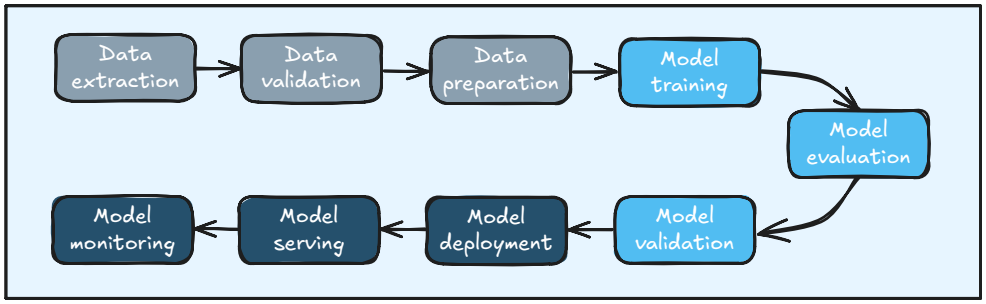
1. Training
2. Evaluation
3. Validation

## Code

1. Deployment

2. Serving
3. Monitoring

*Modularizing things, diagrammatically*



## MLOps Principles

| | | | |
|---|---|---|---|
| Aug-25 | w35 | 1 | Course introduction and overview of mlops |
| Sep-1 | w36 | 2 | [1h] – MLOps project overview<br>[1h] – Git advanced versioning |
| Sep-8 | w37 | 3 | Data and model versioning |
| Sep-15 | w38 | 4 | Golang basic |
| Sep-22 | w39 | 5 | Golang builds, tests, modules, packages |
| Sep-29 | w40 | 6 | Containers and Docker |
| Oct-6 | w41 | 7 | Workflow orchestration |
| Oct-20 | w43 | 8 | Workflow orchestration |
| Oct-27 | w44 | 9 | MLOps projects |
| Nov-3 | w45 | 10 | Productionising models |

## Versioning

- **Data**
    1. Data preparation pipelines
    2. Features store
    3. Datasets
    4. Metadata
- **Model**
    1. ML model training pipeline
    2. ML model (object)
    3. Hyperparameters

4. Experiment tracking

- **Code**
  1. Application code
  2. Configurations

## Automation

- **Data**
  1. Data transformation
  2. Feature creation and manipulation
- **Model**
  1. Data engineering pipeline
  2. ML model training pipeline
  3. Hyperparameter/Parameter selection
- **Code**
  1. ML model deployment with CI/CD
  2. Application build

## Reproducibility

- **Data**
  1. Backup data
  2. Data versioning
  3. Extract metadata
  4. Versioning of feature engineering
- **Model**
  1. Hyperparameter tuning is identical between dev and prod
  2. The order of features is the same
  3. Ensemble learning: the combination of ML models is same
  4. The model pseudo-code is documented
- **Code**
  1. Versions of all dependencies in dev and prod are identical
  2. Same technical stack for dev and production environments

3. Reproducing results by providing container images or virtual machines

## Deployment

- **Data**
  1. Feature store is used in dev and prod environments
- **Model**
  1. Containerization of the ML stack
  2. REST API
  3. On-premise, cloud, or edge
- **Code**
  1. On-premise, cloud, or edge

## Testing

- **Data**
  1. Data Validation (error detection)
  2. Feature creation unit testing
- **Model**
  1. Model specification is unit tested
  2. ML model training pipeline is integration tested
  3. ML model is validated before being operationalized
  4. ML model staleness test (in production)
  5. Testing ML model relevance and correctness
  6. Testing non-functional requirements (security, fairness, interpretability)
- **Code**
  1. Unit testing
  2. Integration testing for the end-to-end pipeline

## Monitoring

- **Data**
  1. Data distribution changes (training vs. serving data)
  2. Training vs serving features
- **Model**

1. ML model decay
2. Numerical stability
3. Computational performance of the ML model

- **Code**
  1. Predictive quality of the application on serving data

## Project Success Factors

# Project success factors

Small diversion before going into details.



## Questions for you

### *How many DS projects fail?*

- My experience: >50%
- VentureBeat 2019: 87%
- Gartner 2018: 85%

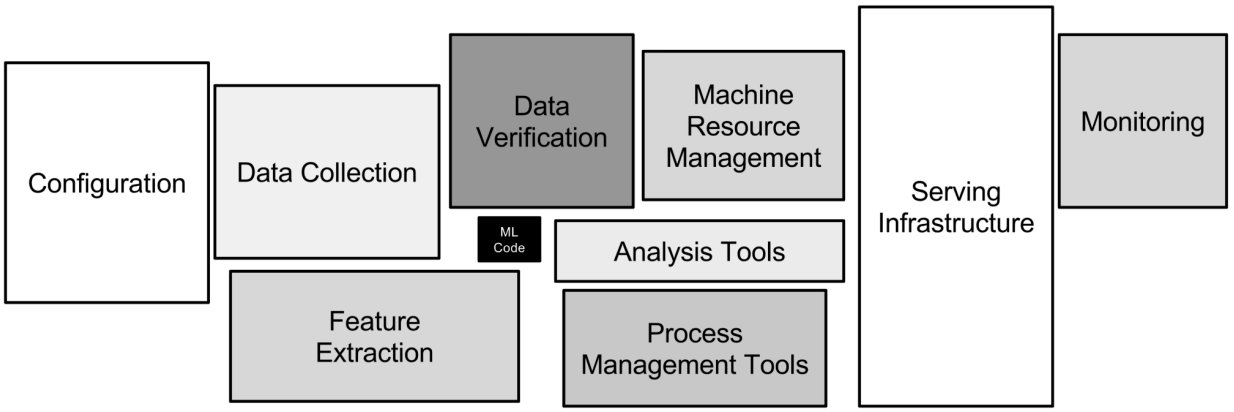
### *What do you think "fail" means?*

- Typically: run in production
- Personal criterion: X users over Y years
- Academic: DS PRO-S

### *Why do you think they fail?*

- Bad data
- Lack of integration
- Lack of diverse expertise
- Unclear business involvement
- Changing anything changes everything

## Tech debt




From https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

- **Complex Models Erode Boundaries**: many various dependencies and consumers

- **Data Dependencies Cost More than Code Dependencies**: unstable, unused and/or legacy data
- **Feedback Loops**: both direct and hidden
- **ML-System Anti-Patterns**: glue code, pipeline jungles, dead experimental paths, abstractions, common smells
- **Configuration Debt**: different data types/sources may have different requirements/validity/configs, etc
- **Dealing with Changes in the External World**: ML systems that can take action affect the world
- **Other Areas of ML-related Debt**: data testing, reproducability, systems with many models, cultural debt

## Organisation Maturity - Overview

# Organisation Maturity



## Frameworks

| Company/Paper | Maturity Level Proposed(s) |
|---|---|
| Google | Level 0: Manual Process<br>Level 1: ML Pipeline Automation<br>Level 2: Automated Training |
| Microsoft | Level 0: No MLOps<br>Level 1: DevOps but No MLOps<br>Level 2: Automated Training<br>Level 3: Automated Model Deployment<br>Level 4: Full MLOps Automated Operations |
| Amazon | Level 1: Initial Phase<br>Level 2: Repeatable Phase<br>Level 3: Reliable Phase<br>Level 4: Scalable Phase |
| IBM | Level 0: No MLOps<br>Level 1: ML Pipeline Automation<br>Level 2: CI/CD Pipeline Integration<br>Level 3: Advanced MLOps |
| Meenu Mary et al. | Level A: Automated Data Collection<br>Level B: Automated Model Deployment<br>Level C: Semi-automated Model Monitoring<br>Level D: Fully-automated Model Monitoring |

From *Navigating MLOps: Insights into Maturity, Lifecycle, Tools, and Careers* (https://arxiv.org/html/2503.15577v1)

## Generic breakdown

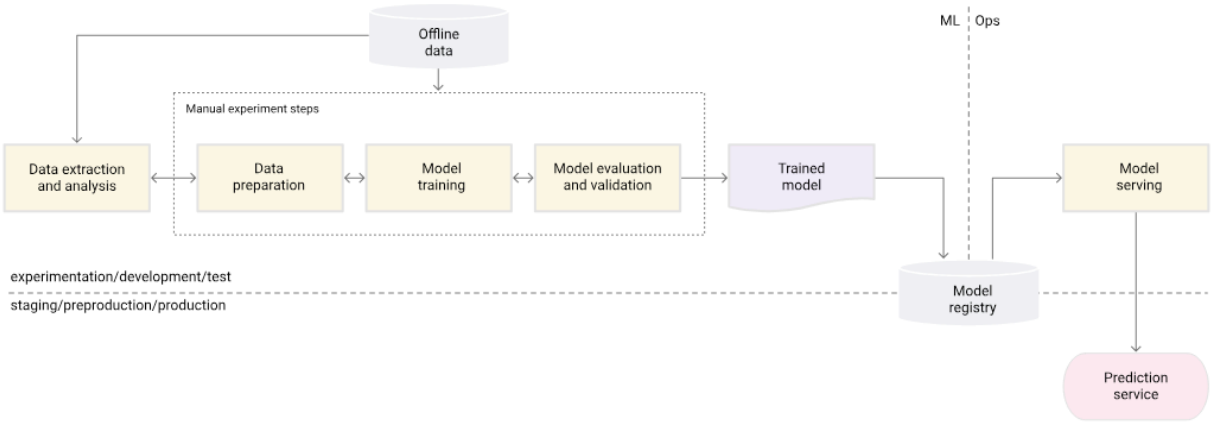
- **Level 0**: No MLOps/manual process
- **Level 1**: Semi-automated data/model/code pipelines
- **Level 2**: Fully automated

## Organisation Maturity - Level 0: Manual Process

# Level 0: Manual Process



## Overview




From https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

- Interactive process (e.g., notebooks)
    - YOU deploy the model
- DS – engineer disconnect
    - Training-serving skew
- No CI/CD
- No monitoring

---

## Dealing with challenges

- Actively monitor quality
- Frequently retrain model
- Continuously experiment

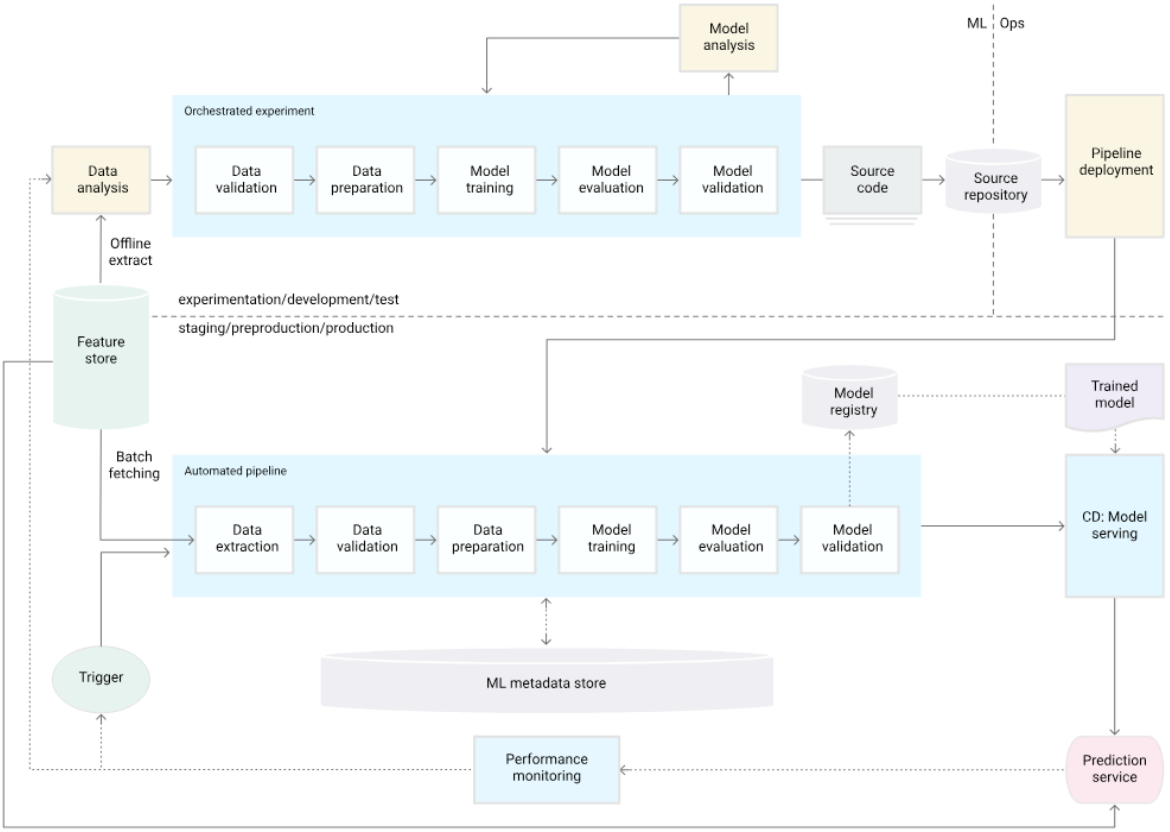
→ ML Automation

## Organisation Maturity - Stage 1: Data/Model/Code

# Level 1: Data/model/code semi-automation



## Overview




From https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

- Rapid experiment
- Continuous model delivery/testing
- Modularized and re-used code
    - No training-serving skew
- Whole ML pipeline deployments

## Dealing with breaking models



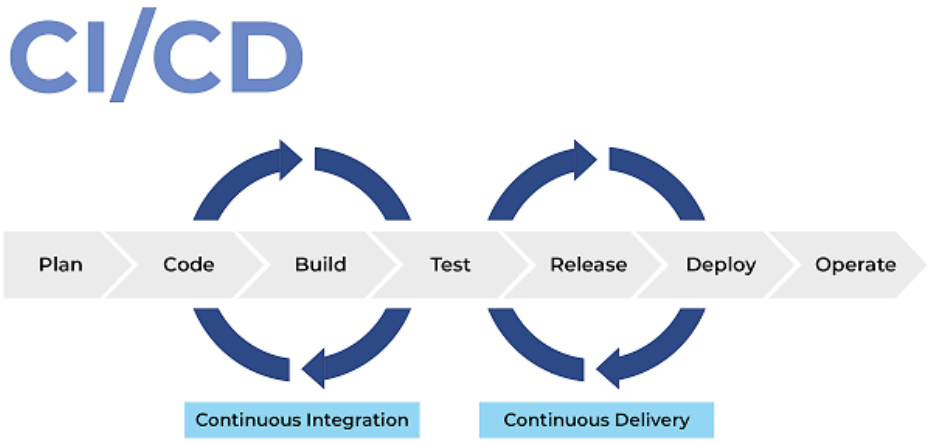
- Data validation
  - Data schema skews
  - Data values skews
- Model validation
- Offline vs. online
- Evaluation metric
- “DevOps”

→ Pipeline triggers next lecture

# Level 2: Fully automated



## Overview




From https://www.linkedin.com/pulse/cicd-what-why-important-rob-larter/

Continuous...

... integration (pipelines)

... delivery (pipelines, models)

## Further drill down

| CATEGORY | LEVEL 0 | LEVEL 1 | LEVEL 2 | LEVEL 3 | LEVEL 4 |
|---|---|---|---|---|---|
| STRATEGY | – No data scientists hired<br>– Skeptical of value of ML among an executive team | – Small and siloed data science and data engineering teams<br>– A small number of ML champions in an executive team | – Small data science, data engineers and software development teams started to be coordinated<br>– Small data science, data engineers and software development teams started to be coordinated | – Large, well-integrated teams across data science, engineering and software development<br>– Chief Data Officer, and C-suite level sponsoship exists<br>– New team members brought up to speed in weeks<br>– Project checkpoints to ensure ML is considered for major projects | – ML seen as strategic, driving company initiatives<br>– Well-governed process for ML delivery<br>– Engineers & researchers are embedded on the same team |
| ARCHITECTURE | – Data silos with one-off integration<br>– Data not prepared nor ready for ML | – Basic enterprise data ready for ML<br>– Data architecture still immature<br>– Tacit commitment to meaningful enterprise data in the cloud | – Data architecture is mature<br>– Most enterprise data ready for ML in the cloud<br>– Overt commitment to the cloud | – Enterprise data is well cataloged and managed<br>– Automated data pipelines in place<br>– ML configuration and infrastructure is managed<br>– ML models automatically provisioned as microservices | – Comprehensive architecture to effectively govern all data<br>– Consistent data storage and consumption pipeline across projects<br>– Target ML infrastructure monitored for cost-effectiveness and optimal utilization |
| MODELING | – Manual process for model training<br>– Limited pilot studies | – Manual ML model training and live pilots<br>– Basic experiment tracking, no model management | – Experiment tracking and model management in place<br>– Models dependencies not well understood | – Models cataloged through lifecycle, supporting reproducibility and reuse<br>– Models dependencies not well understood<br>– Output from ML is predictable and consistent, with auditable and reproducible outcomes | – Interdependencies between models are monitored and managed<br>– Impact of small changes to ML models can be measured |
| PROCESSES | – No DevOps practices adopted<br>– No clearly defined success criteria for ML projects | – DevOps practices like CI/CD have been adopted for non-ML components<br>– No consistency in measures for ML or MLOps success | – Development iterative but CI/CD not in place for models<br>– ML infrastructure expertise not broadly available<br>– ML configuration is an afterthought<br>– Metrics/measures in place but not consistent across projects | – Data tested for model applicability and monitored for changes in distribution<br>– All artifacts (data sets, tests, models) under version control<br>– DevOps practices like CI/CD, code reviews in place for ML code<br>– Production MLOps pipeline flow includes packaging, deployment, serving and operational monitoring | – Comprehensive MLOps pipeline supporting frequent model updates<br>– New algorithmic approaches can be tested at full scale<br>– Automatic metrics gathering, alerts, issues analysis (such as data drift) and automated retraining of systems is in place |
| GOVERNANCE | – Not considered | – Not considered, though the organization may have broader views<br>– No notion of the concept of bias in models | – Model explainability not considered<br>– Models may harbor prediction bias<br>– Model releases are tracked | – Security policies applied to models, data<br>– Ethics and explainability consideration for models and ML systems<br>– Good faith attempts to remove biased variables from models<br>– Potential for malicious use of ML considered in ML lifecycle | – Cybersecurity experts engaged in ML operations<br>– ML systems protected from external manipulation<br>– End-to-end audit trail for ML - who, why, when |

From https://gigaom.com/report/delivering-on-the-vision-of-mlops/
https://medium.com/@NickHystax/mlops-maturity-levels-the-most-well-known-models-5b1de94ea285

- Strategy

- Architecture
- Modeling
- Process
- Governance

---

## My company? In my opinion...

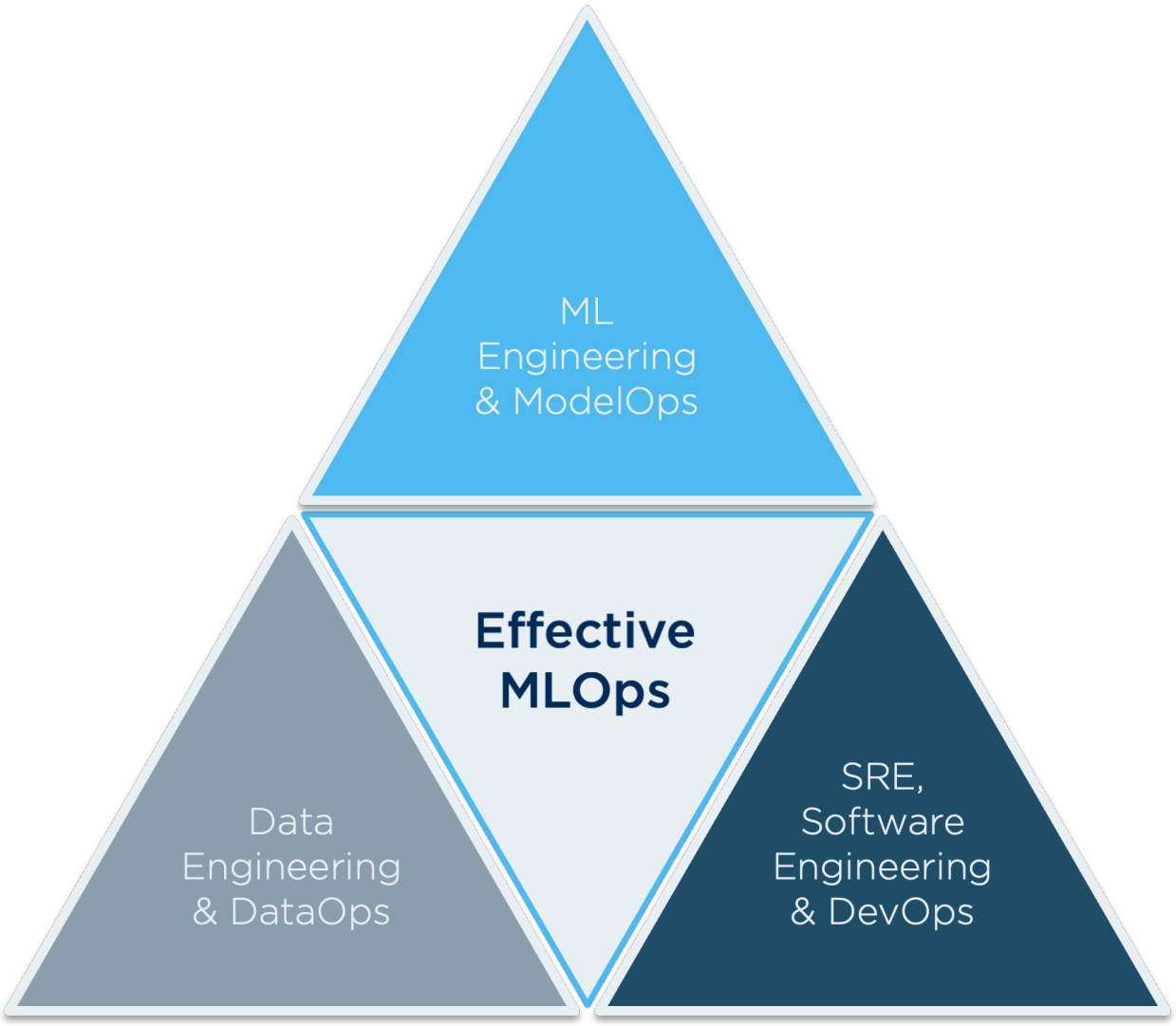
| Strategy | Architecture | Modeling | Process | Governance |
|---|---|---|---|---|
| Level 1-4 | Level 2-4 | Level 0-4 | Level 1-4 | Level 1-4 |
| (4) ML seen as strategic driving company initiatives<br>(1) Small and siloed DS and DE teams<br>(3) CDO + C-level sponsorship | (4) Target ML infra monitored for cost effectiveness<br>(2) Data architecture is mature<br>(3) ML config and infrastructure is managed | (3) Models catalogued through lifecycle<br>(1) Manual ML model training<br>(2) Model dependencies not well understood | (4) Automatic metrics gathering (data drift, etc)<br>(1) DevOps practices like CI/CD for non-ML things<br>(3) All artifacts under version control | (4) End-to-end audit trail for ML<br>(1) Not considered though org may have broader views<br>(3) Ethics consideration for models and ML systems |

## Three Components of ML Software

# Three Components of ML Software

Based on https://ml-ops.org/content/three-levels-of-ml-software

## Our beloved triangle




From https://ml-architects.ch/blog_posts/mlops_maturity_model.html

## Same idea in Cookiecutter for Data Science

- `0.01` - Helps leep work in chronological order. The structure is `PHASE.NOTEBOOK`. `NOTEBOOK` is just the Nth notebook in that phase to be created. For phases of the project, we generally use a scheme like the following, but you are welcome to design your own conventions:
  - `0` - Data exploration - often just for exploratory work
  - `1` - Data cleaning and feature creation - often writes data to `data/processed` or `data/interim`
  - `2` - Visualizations - often writes publication-ready viz to `reports`
  - `3` - Modeling - training machine learning models
  - `4` - Publication - Notebooks that get turned directly into reports
- `pjb` - Your initials; this is helpful for knowing who created the notebook and prevents collisions from people working in the same notebook.
- `data-source-1` - A description of what the notebook covers

From https://cookiecutter-data-science.drivendata.org/using-the-template/

- **Data**: 0 - 2
- **Model**: 2 - 3
- **Code**: 4

## Data Component

# Data Component

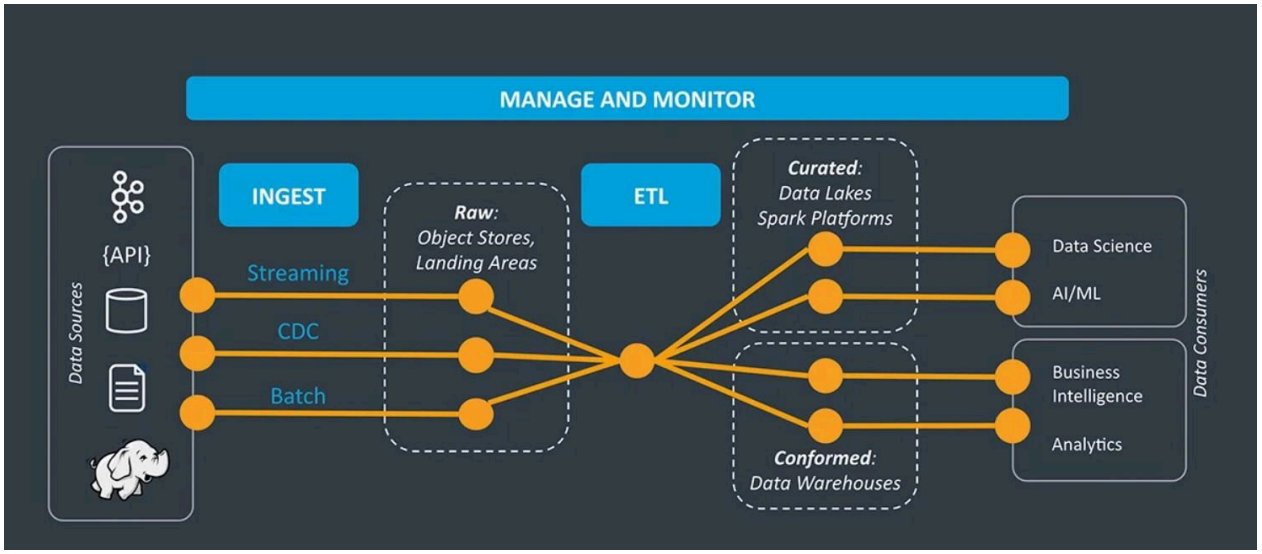
## Garbage in, garbage out



From Shutterstock, because I couldn't find a good non-AI image.

---

## Data extraction/ingestion

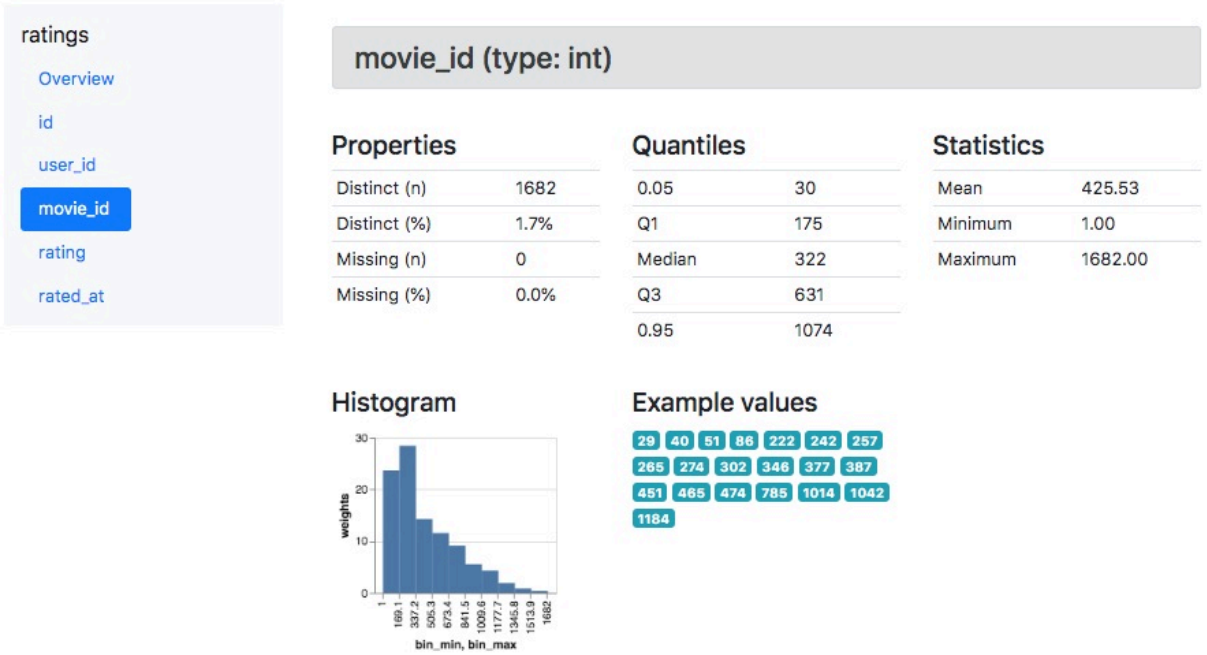



From https://firsteigen.com/blog/10-data-ingestion-tools-to-fortify-your-data-strategy/

- Determine sources
- Compliance/sensitive data
- Test data split

## Data exploration and validation

## Data exploration and validation




From https://legacy.017.docs.greatexpectations.io/docs/0.14.13/reference/data_discovery/

- How does the data look like?
  - Mean, std
  - Data types
  - Missing values
- Useful for alerts

## Data preparation/wrangling/cleaning

## Data preparation/wrangling/cleaning

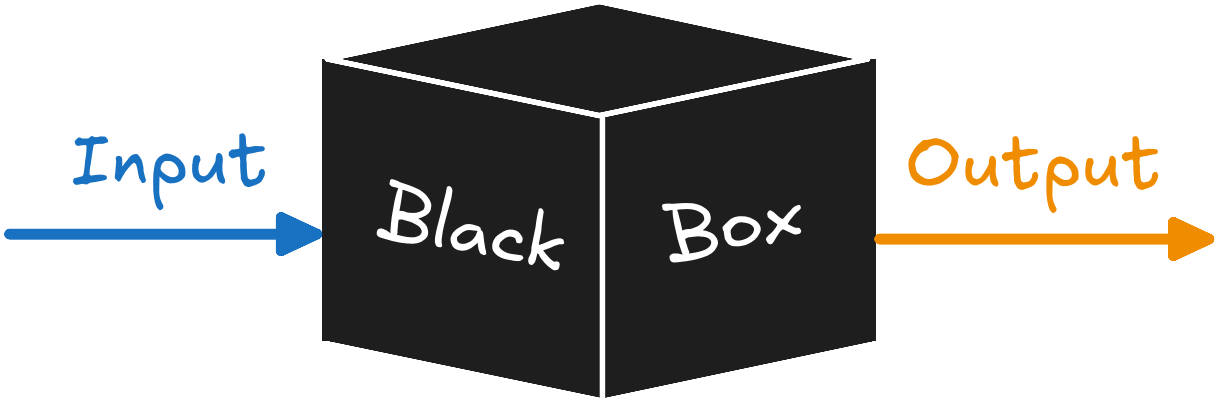
- Transformations
  - Fill missing values
  - Remove outliers
  - Remove irrelevant data

- “Silver” layer

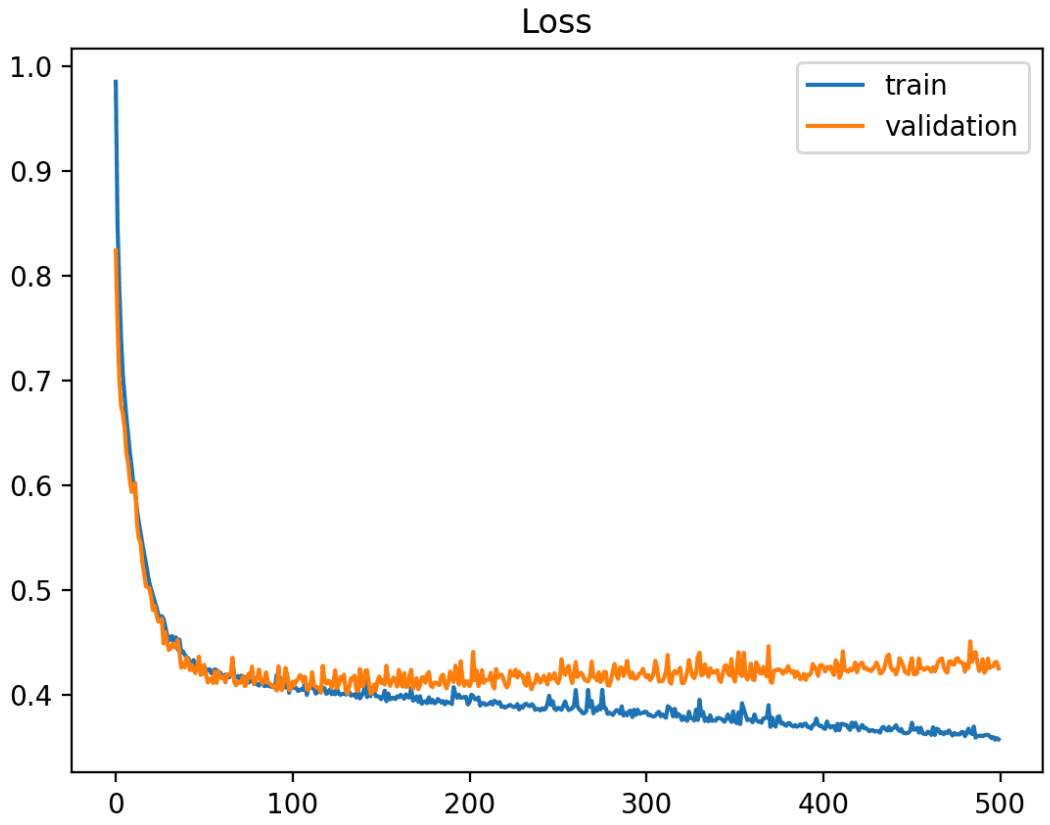
### Model Component

## Model Component

### Black box



## Model training



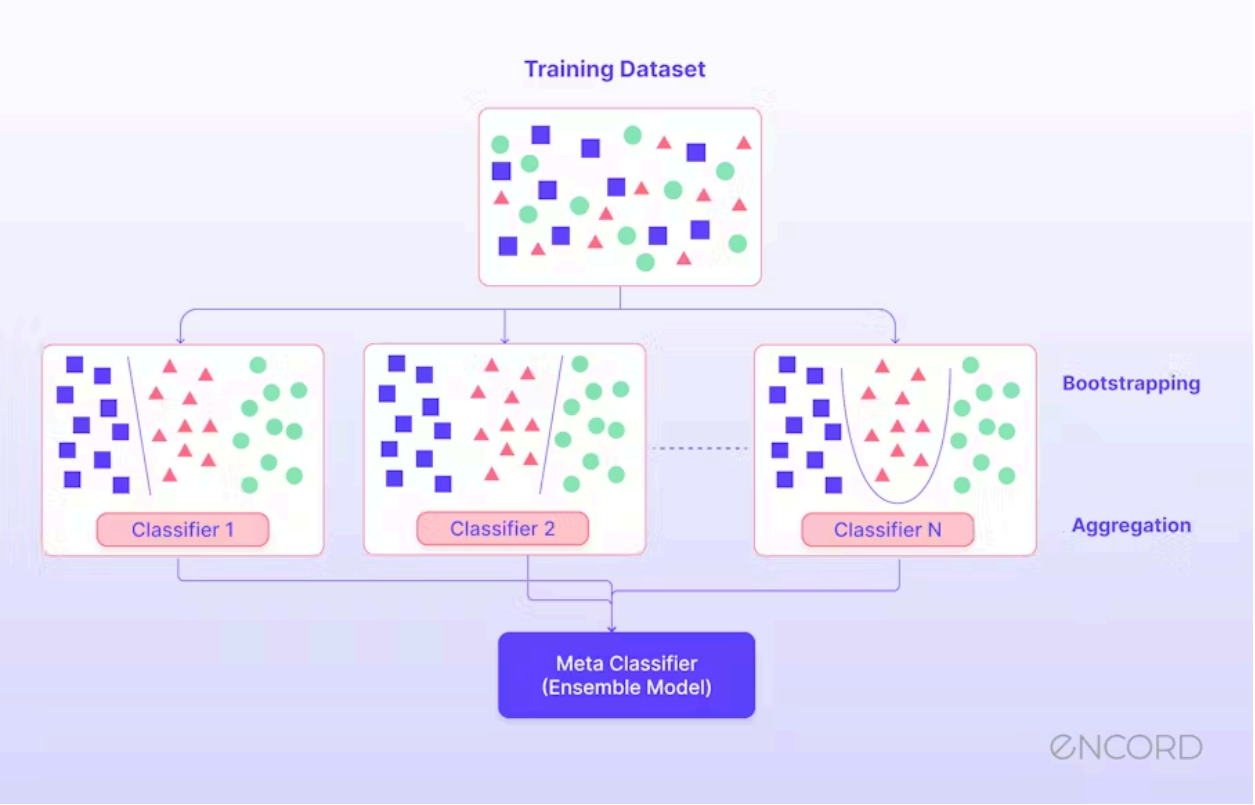

From https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/

- Don’t get too attached to one model
- Feature + model engineering
- Random search for hyperparameters

→ Robustness is important

## Model evaluation/testing

## Model evaluation/testing




From https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/

- Holdout test set
- Multiple metrics to measure…
  - Error
  - Bias
- Must be better than baseline

## Model validation/packaging

| Format | Open-Format | Vendor | File Extension | License | ML Tools & Platforms Support | Human-readable | Compressior |
|---|---|---|---|---|---|---|---|
| "almagnation" | - | - | - | - | - | - | ✅ |
| PMML | ✅ | DMG | .pmml | AGPL | R, Python, Spark | ✅ (XML) | ❌ |
| PFA | ✅ | DMG | JSON | | PFA-enabled runtime | ✅ (JSON) | ❌ |
| ONNX | ✅ | SIG LFAI | .onnx | | TF, CNTK, Core ML, MXNet, ML.NET | ❌ | ✅ |
| TF Serving Format | ✅ | Google | .pf | | Tensor Flow | ❌ | g-zip |
| Pickle Format | ✅ | | .pkl | | scikit-learn | ❌ | g-zip |
| JAR/POJO | ✅ | | .jar | | H2O | ❌ | ✅ |
| HDF | ✅ | | .h5 | | Keras | ❌ | ✅ |
| MLEAP | ✅ | | .jar/.zip | | Spark, TF, scikit-learn | ❌ | g-zip |
| Torch Script | ❌ | | .pt | | PyTorch | ❌ | ✅ |
| Apple .mlmodel | ❌ | Apple | .mlmodel | | TensorFlow, scikit-learn, Core ML | - | ✅ |

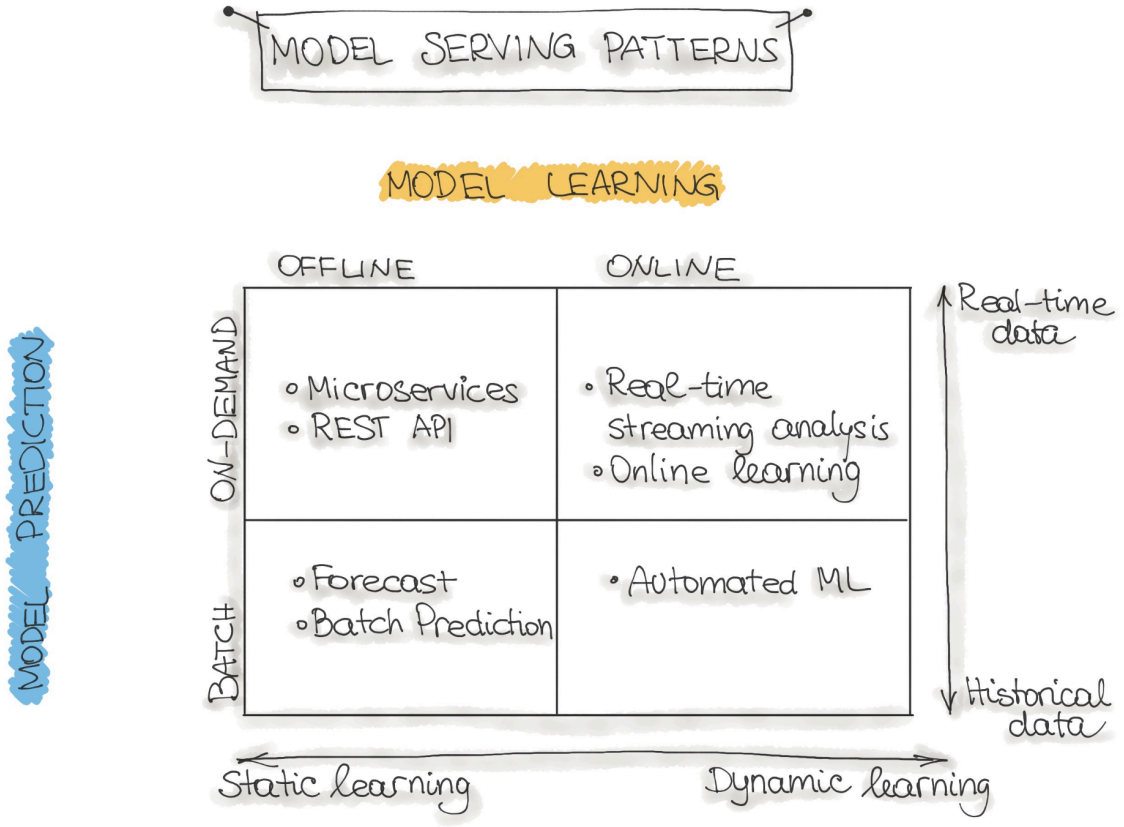
Adapted from https://ml-ops.org/content/three-levels-of-ml-software#model-machine-learning-pipelines

- Unit/integration testing
- Different models, different save formats

## Code Component

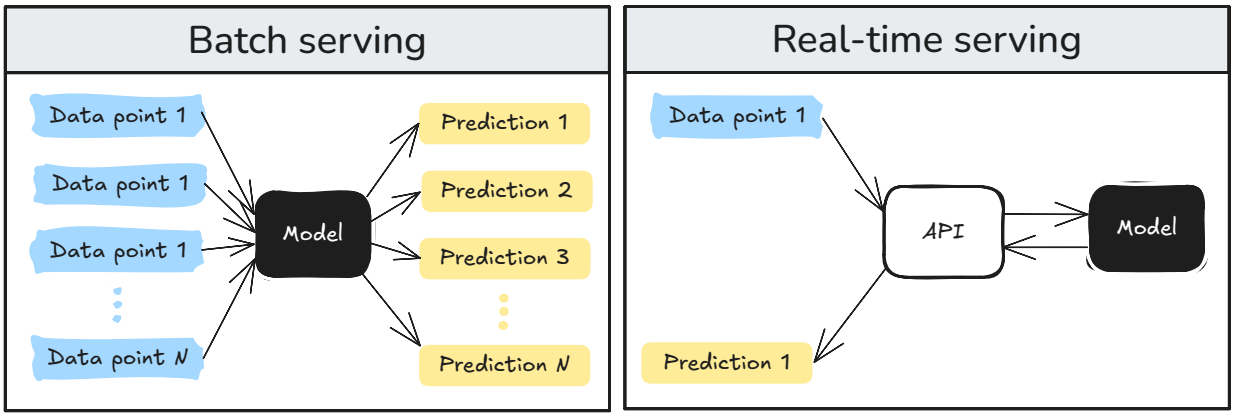
# Code Component

## Different use-cases, different patterns




From https://ml-ops.org/content/three-levels-of-ml-software

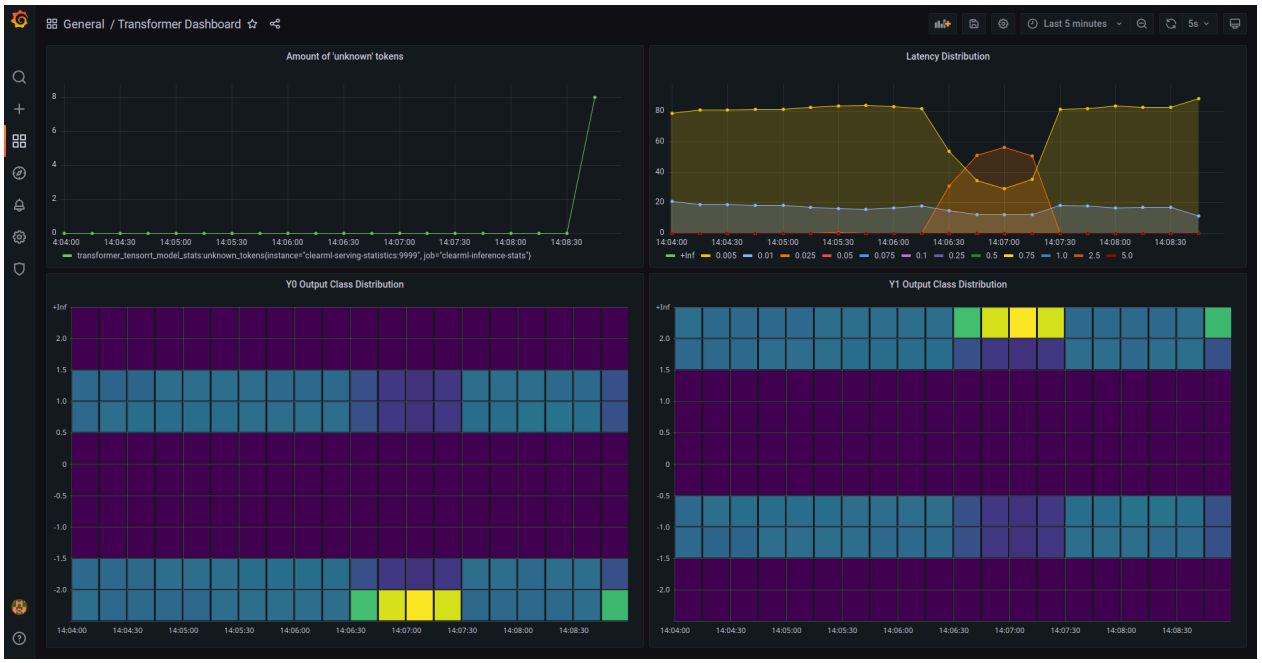
## Model serving




Adapted from https://www.iguazio.com/glossary/model-serving-pipeline/

- First training, then "inference"
- Modes of serving
  - In batch
  - On demand
- Other consideration
  - Sanity/confidence check input/output
  - Serving to humans or machines?
  - Fallback methods
  - Interpreting output outside of model

## Model logging/monitoring




From https://grafana.com/blog/2023/08/18/monitoring-machine-learning-models-in-production-with-grafana-and-clearml/

- Log predictions
- Performance deteriorates
- Data changes
- Abuse/adversarial

→ More next lesson

## Model maintenance

- Model freshness
- Retraining time
- Deployment cost



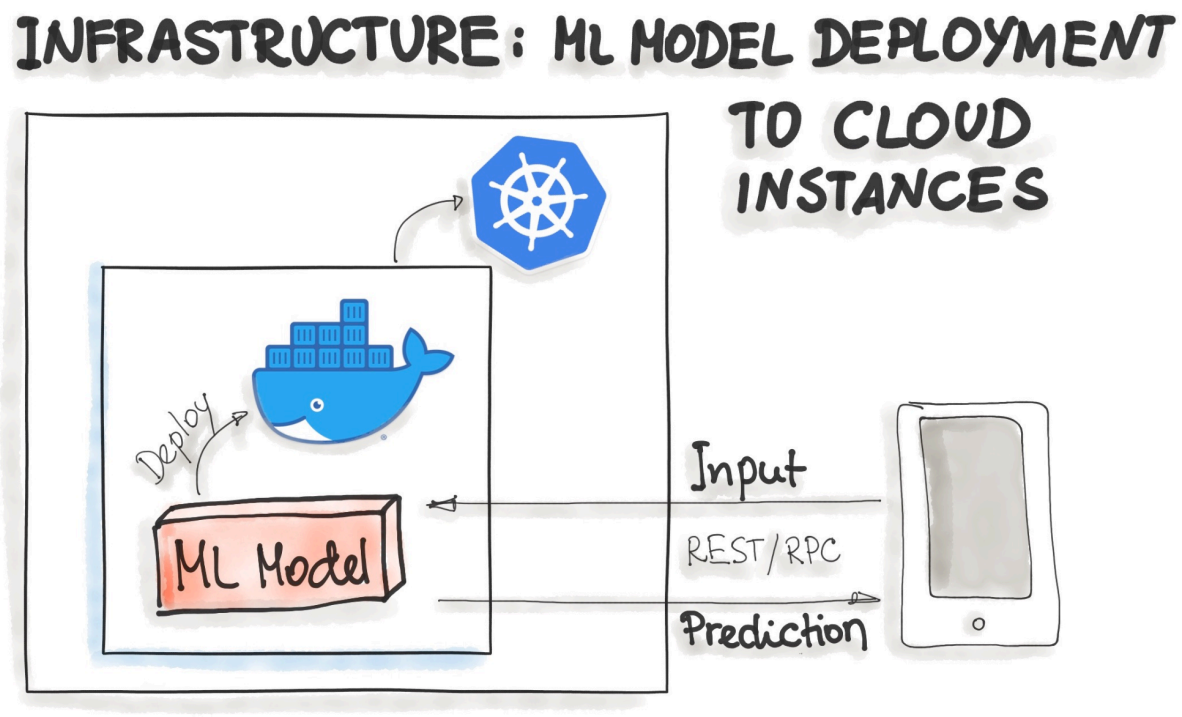
## Course project

*What pattern does that follow?*

- Offline training
- On-demand through REST API

# Deployment Strategies

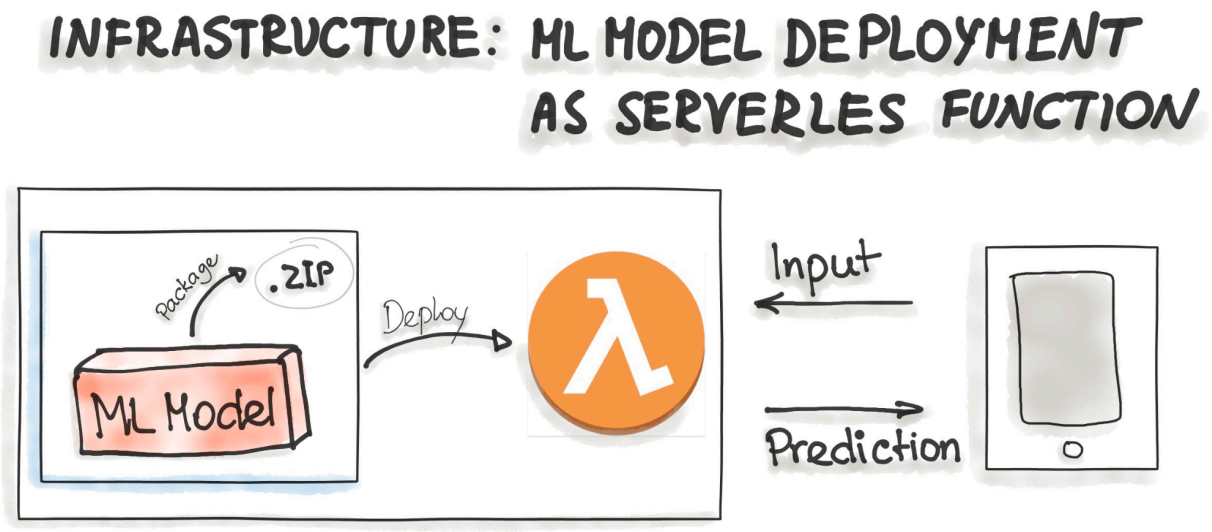
## Docker containers




From https://ml-ops.org/content/three-levels-of-ml-software

- Lightweight
- More in Week 9

## Serverless functions




- Does not require running instance
- More expensive per inference

## Summary

# Summary

1. **DevOps vs MLOps**
    - ML + Dev + Ops
2. **MLOps Principles**
    - Versioning
    - Automation
    - Reproducability
    - Deployment
    - Testing
    - Monitoring
3. **MLOps Components**
    - Data
    - Model

- Code

4. **MLOps in Practice**
    - Success factors
    - Company maturity levels (0-2)

---

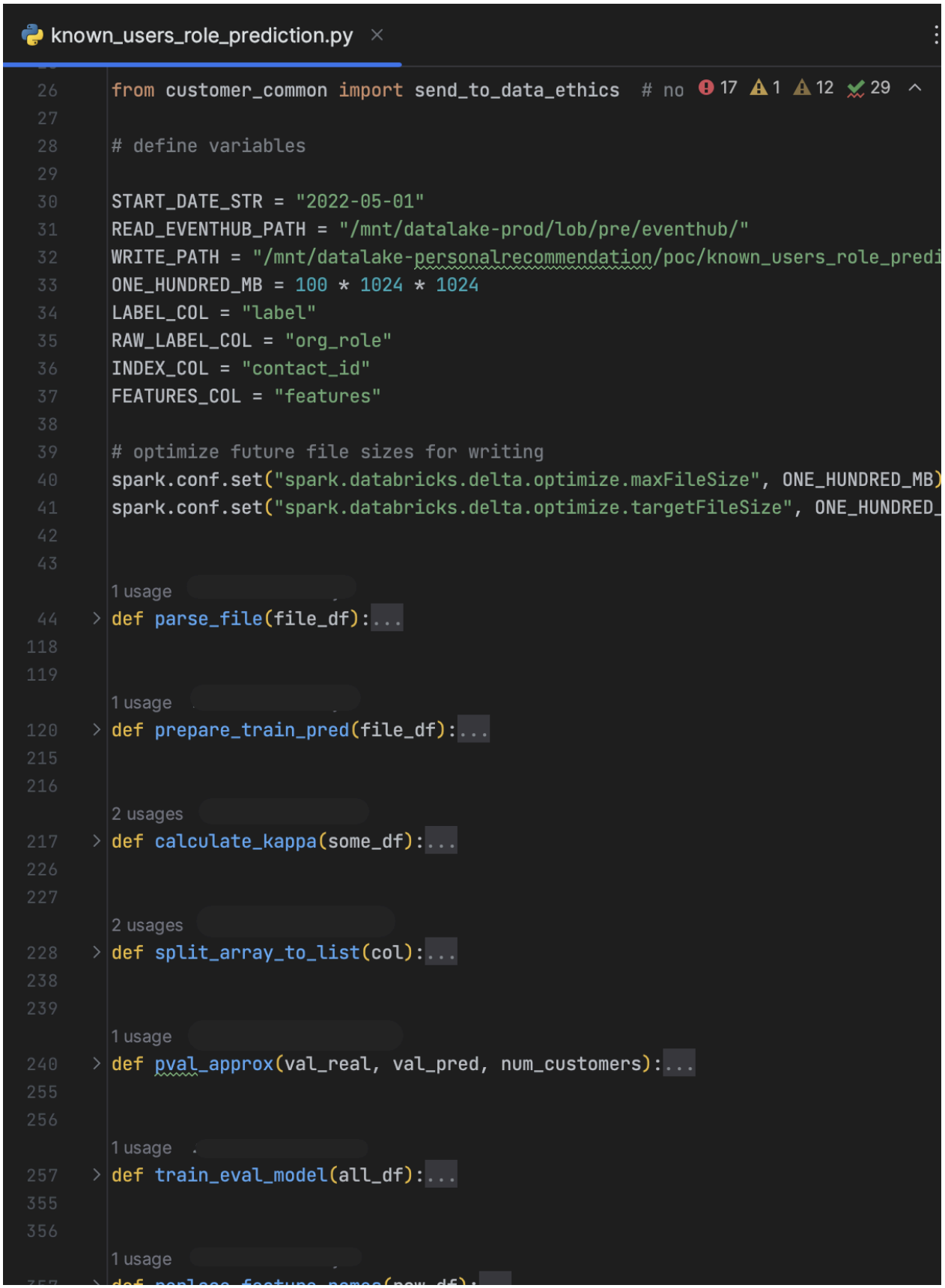
## Next lessons

---

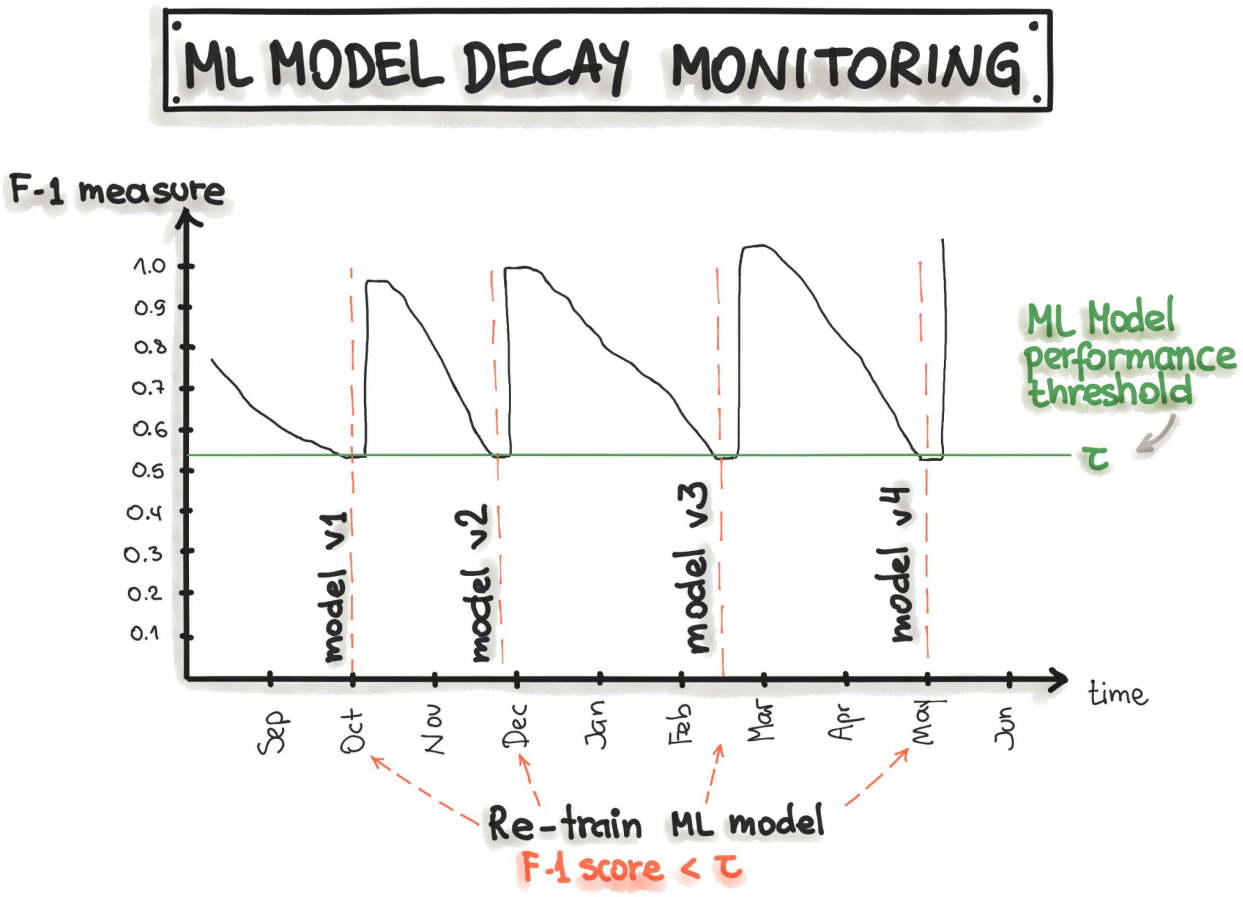
### Docker

**Lecture + Exercises**: What is Docker, how do you build

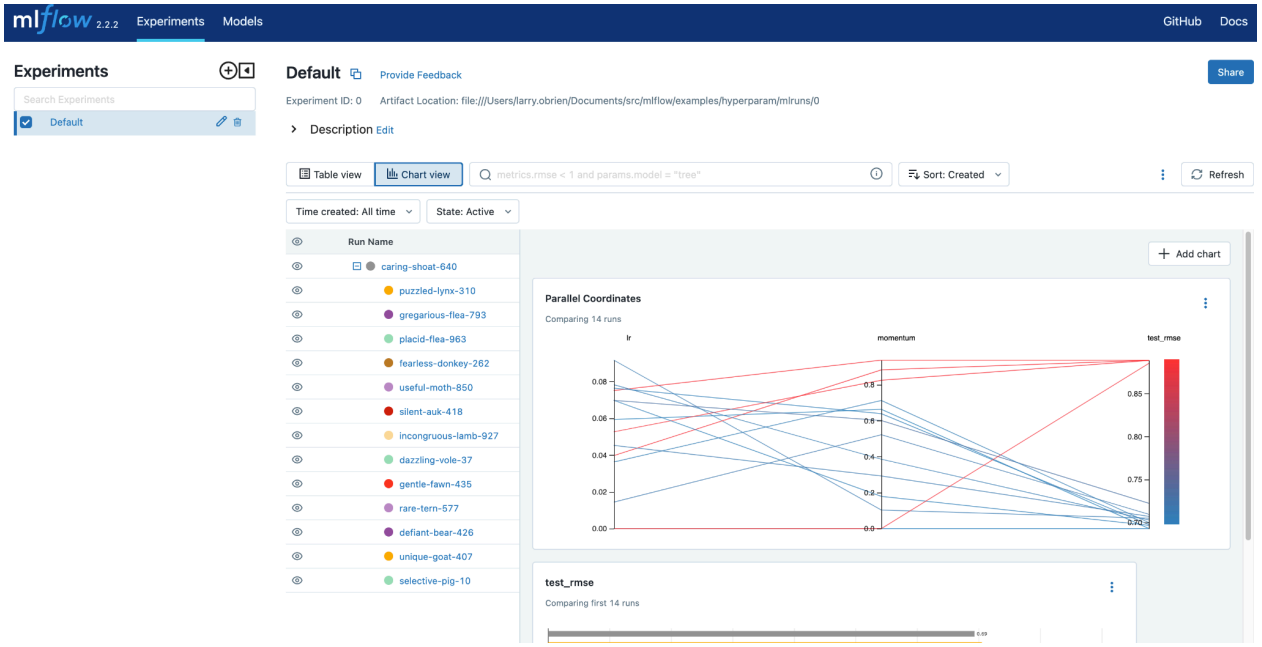
## Productionising models


known_users_role_prediction.py

```
26  from customer_common import send_to_data_ethics  # no  17  1  12  29
27
28  # define variables
29
30  START_DATE_STR = "2022-05-01"
31  READ_EVENTHUB_PATH = "/mnt/datalake-prod/lob/pre/eventhub/"
32  WRITE_PATH = "/mnt/datalake-personalrecommendation/poc/known_users_role_predi
33  ONE_HUNDRED_MB = 100 * 1024 * 1024
34  LABEL_COL = "label"
35  RAW_LABEL_COL = "org_role"
36  INDEX_COL = "contact_id"
37  FEATURES_COL = "features"
38
39  # optimize future file sizes for writing
40  spark.conf.set("spark.databricks.delta.optimize.maxFileSize", ONE_HUNDRED_MB)
41  spark.conf.set("spark.databricks.delta.optimize.targetFileSize", ONE_HUNDRED_
42
43
      1 usage
44  def parse_file(file_df):...
118
119
      1 usage
120 def prepare_train_pred(file_df):...
215
216
      2 usages
217 def calculate_kappa(some_df):...
226
227
      2 usages
228 def split_array_to_list(col):...
238
239
      1 usage
240 def pval_approx(val_real, val_pred, num_customers):...
255
256
      1 usage
257 def train_eval_model(all_df):...
355
356
      1 usage
```




**Lecture**: DORA metrics, monitoring and maintenance, what does good look like.




**Exercises**: Use MLflow to log artifacts and ML experiments across, advanced MLflow.

## Resources

- ML-Ops.org: Motivation
- ML-Ops.org: MLOps principles
- ML-Ops.org: Three levels of ML Software
- ML-Ops.org: End-to-End ML
- Google & MLOps levels
- ML Engineering, Chapter 2 (http://bit.ly/MLEbook-Chapter2)

## Project Success

# Project Success

Recap of "Project Success Factors" and "Tech Debt".

## Discussion

### What does it take for a data science project to be successful? Can you agree on a definition of "success"?

- Typically: the model actually runs in production
- Personal criterion: X users over Y years
- Academic: DS PRO-S

## Questions

How many data science projects never make it into production?

- A: Roughly 13 out of 100
- B: Roughly 87 out of 100
- C: Almost all of them

- D: Roughly 50 out of 100

## Answer

**B.** VentureBeat 2019: 87% never make it into production, and Gartner 2018: 85% deliver erroneous outcomes. My own experience is >50%.

A is actually the *success* rate (100 - 87). C overstates it - most projects fail, but not almost all. D is closer to my personal experience but still too optimistic compared to the published numbers.




---

What does "fail" typically mean in these statistics?

- A: The model crashes every time it is called in production
- B: The team behind the project was disbanded
- C: The model never runs in production
- D: The model scored below 50% accuracy

## Answer

**C.** The statistics count everything that never reaches production - including projects cancelled before deployment.

A, B, and D are specific failure modes, but they are too narrow: a project that never shipped at all still counts as a failure here.

---

Why do data science projects fail? Select all that apply.

- A: Bad data
- B: Not enough GPUs
- C: Lack of integration into existing systems
- D: Unclear business involvement
- E: The model was not a large language model
- F: Lack of diverse expertise
- G: Changing anything changes everything

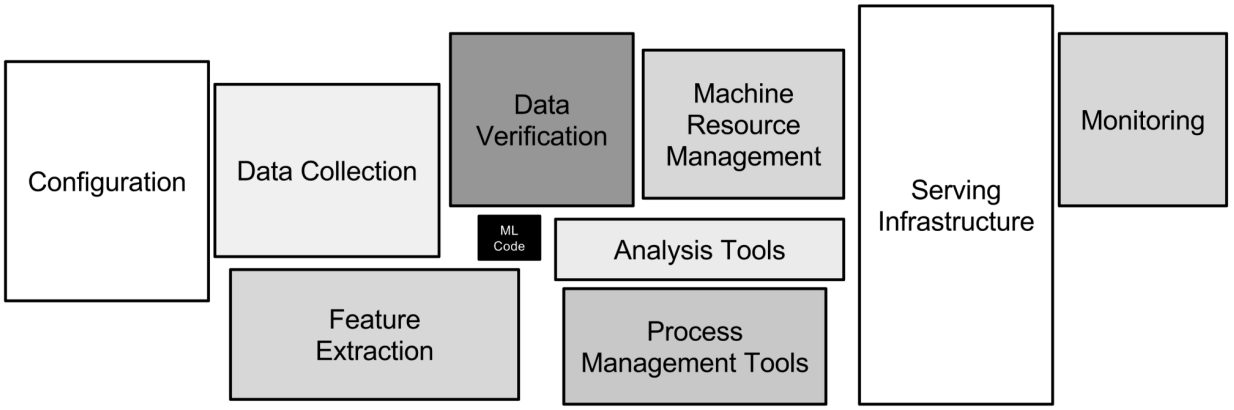
### Answer

**A, C, D, F, and G.** These are the recurring themes: bad data, no integration path, no business ownership, one-sided teams, and tight coupling everywhere.

B and E are red herrings - the choice of model family or the amount of hardware is rarely what kills a project.

---

## Tech debt




From https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

The NeurIPS paper "Hidden Technical Debt in Machine Learning Systems" names several anti-patterns. Which of the following are from the paper? Select all that apply.

- A: Glue code

- B: Too many unit tests
- C: Pipeline jungles
- D: Too many CPUs
- E: Dead experimental paths
- F: Configuration debt
- G: Hidden feedback loops

### Answer

**A, C, E, F, and G** are all named in the paper, together with unstable data dependencies and cultural debt.

B and D are not in the paper - having too many tests or too much hardware is not a form of technical debt the authors describe.

---

What is a "pipeline jungle"?

- A: A pipeline that runs in a cloud provider's "jungle" region, i.e., an unsupported zone
- B: A pipeline assembled ad-hoc from whatever tools were at hand, often around legacy data, that nobody dares to touch
- C: A pipeline with so many unit tests that it can never fail in production
- D: A data pipeline that only processes jungle imagery, such as satellite photos

### Answer

**B.** The paper describes pipeline jungles as scar tissue from gluing tools together ad-hoc, typically around legacy data sources.

A and D are jokes, and C confuses testing with pipeline design - the problem is the structure, not the lack of tests.

## Organisation Maturity

# Organisation Maturity

Recap of "Organisation Maturity - Overview" and the maturity stages.

## Discussion

### How would you assess the MLOps maturity of a company? Can you mention any specific examples?

Typical things to look for, mapping to the generic breakdown:

- Is training a manual, interactive process or an automated pipeline?
- Who deploys the model - a person or a pipeline?
- Is there CI/CD, versioning, monitoring?
- **Level 0**: No MLOps/manual process
- **Level 1**: Semi-automated data/model/code pipelines
- **Level 2**: Fully automated

---

## Questions

CocoAnalytics is a four-person fintech startup. One data scientist trains a fraud-detection model in a Jupyter notebook on a company laptop, using data exported manually from the bank. The trained model is a pickle file that is emailed to the backend developer, who wires it into the API by hand. Nobody looks at the model again after release - the first sign of trouble is a customer complaint.

What maturity level is CocoAnalytics at?

- A: Level 1 - Semi-automated data/model/code pipelines
- B: Level 2 - Fully automated
- C: Level 0 - No MLOps/manual process

### Answer

**C.** Interactive (notebook) training, a person deploys the model, DS-engineer disconnect → risk of training-serving skew, no CI/CD, no monitoring.

A is wrong because nothing at all is automated - no pipeline exists. B is wrong because there is no automation and no monitoring loop.

---

StreamLine Logistics, a mid-size courier company, versions data with DVC. A scheduled pipeline runs weekly: it pulls fresh data, validates it, retrains the model, and logs everything to MLflow. When a new model beats the current one, the team reviews the run together, and a developer manually deploys the new model container on Monday morning. There is a Grafana dashboard, but deciding *whether* to retrain is always a human decision.

What maturity level is StreamLine Logistics at?

- A: Level 1 - Semi-automated data/model/code pipelines
- B: Level 0 - No MLOps/manual process
- C: Level 2 - Fully automated

## Answer

**A.** The automated ML pipeline gives rapid experimentation, continuous training and testing, and modularized code → no training-serving skew.

B is wrong because the training pipeline is clearly automated. C is wrong because humans deploy the model and decide on retraining.

---

Nordbank, a large bank, runs its credit-risk models on a fully automated loop: drift detected in production automatically triggers data validation, retraining, and model validation; if the new model passes, it is deployed as a canary without anyone touching it. Everything is monitored in Grafana, and there is an end-to-end audit trail for every model.

What maturity level is Nordbank at?

- A: Level 1 - Semi-automated data/model/code pipelines
- B: Level 0 - No MLOps/manual process
- C: Level 2 - Fully automated

## Answer

**C.** CI/CD for the ML pipelines themselves: production data and model changes automatically trigger pipeline building, testing, and deployment.

A and B are wrong because there is no human in the loop - retraining, validation, and deployment all happen automatically.

---

TechStyle, an e-commerce company, has excellent DevOps: GitHub Actions, Docker, and Kubernetes run the webshop with smooth CI/CD. Machine learning is handled by one data scientist who trains a churn model in a notebook twice a year; the model is manually plugged into a nightly batch job.

What maturity level is TechStyle at?

- A: Level 0, no matter which framework you use
- B: It depends on the framework: Level 0 by the generic breakdown, but "DevOps but no MLOps" (their Level 1) in Microsoft's
- C: Level 2, because their DevOps is excellent

- D: Level 1, no matter which framework you use

### Answer

**B.** The generic breakdown only looks at the ML pipelines, which are manual → Level 0. Microsoft explicitly has a level for this situation: "Level 1: DevOps but No MLOps".

A and D are each too absolute - the frameworks genuinely disagree about where "some DevOps, no MLOps" lands. C is wrong because DevOps maturity does not transfer to ML.

| Company/Paper | Maturity Level Proposed(s) |
|---|---|
| Google | Level 0: Manual Process<br>Level 1: ML Pipeline Automation<br>Level 2: Automated Training |
| Microsoft | Level 0: No MLOps<br>Level 1: DevOps but No MLOps<br>Level 2: Automated Training<br>Level 3: Automated Model Deployment<br>Level 4: Full MLOps Automated Operations |
| Amazon | Level 1: Initial Phase<br>Level 2: Repeatable Phase<br>Level 3: Reliable Phase<br>Level 4: Scalable Phase |
| IBM | Level 0: No MLOps<br>Level 1: ML Pipeline Automation<br>Level 2: CI/CD Pipeline Integration<br>Level 3: Advanced MLOps |
| Meenu Mary et al. | Level A: Automated Data Collection<br>Level B: Automated Model Deployment<br>Level C: Semi-automated Model Monitoring<br>Level D: Fully-automated Model Monitoring |

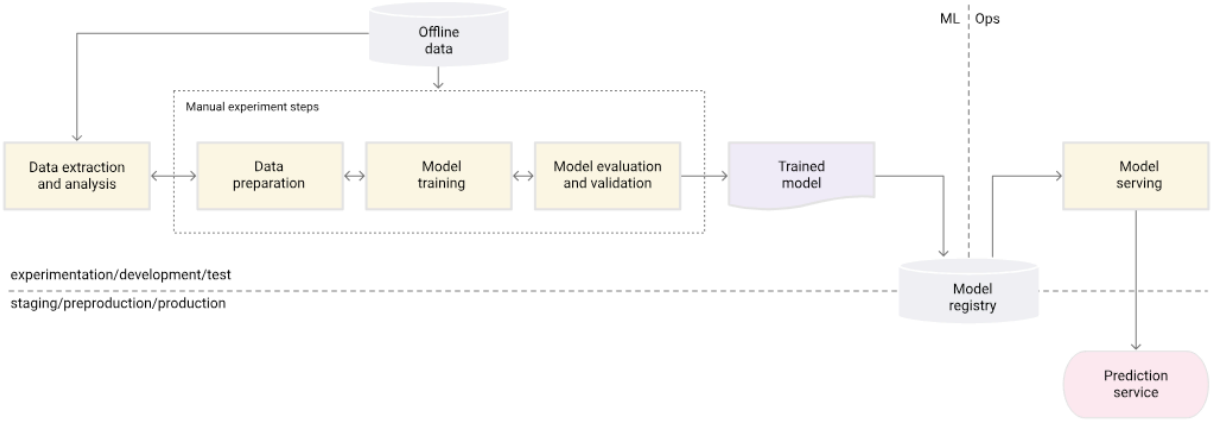
From *Navigating MLOps: Insights into Maturity, Lifecycle, Tools, and Careers* (https://arxiv.org/html/2503.15577v1)

## Architecture and Stages

# Architecture and Stages

Recap of the maturity stage lectures (Level 0, 1, and 2).

## Discussion




From https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

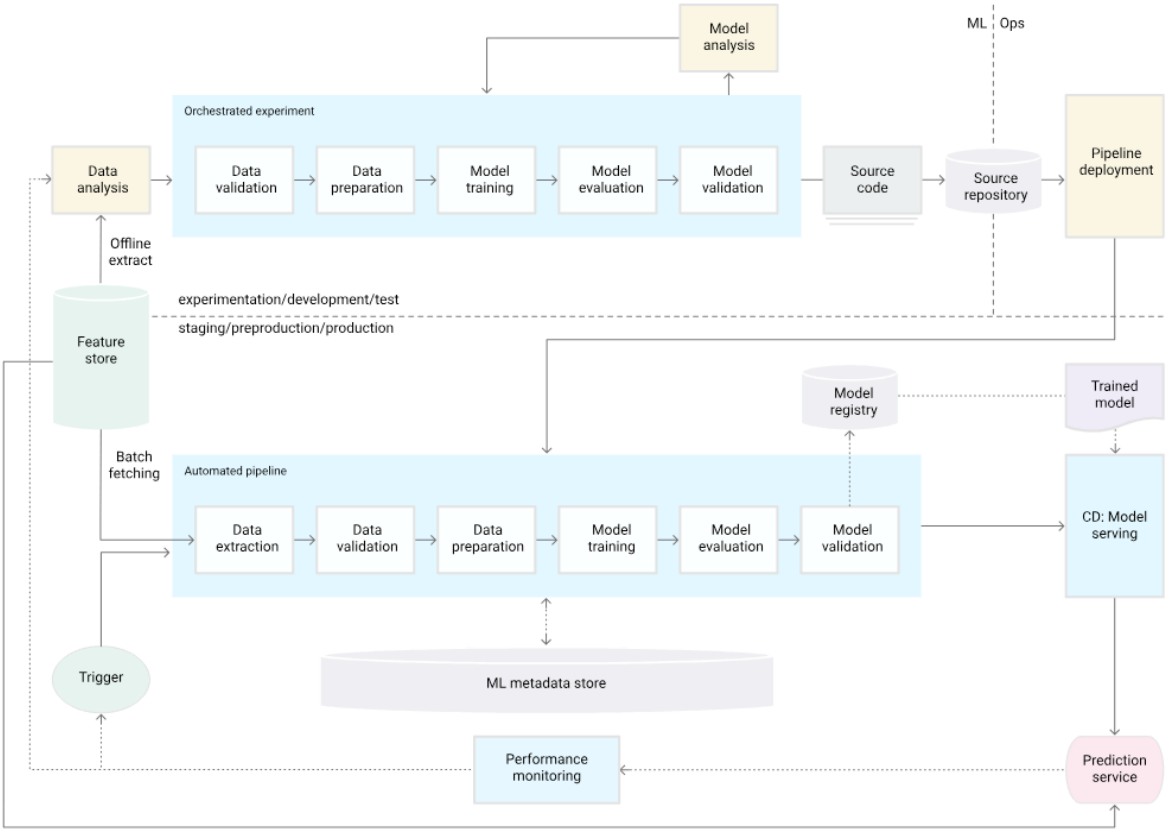
This is the full ML architecture of a real company. What problems will they run into?

### Discussion points

- Interactive process (e.g., notebooks) - YOU deploy the model
- DS - engineer disconnect → training-serving skew
- No CI/CD, no monitoring

→ This is the Level 0 "manual process" architecture

## Questions




From https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

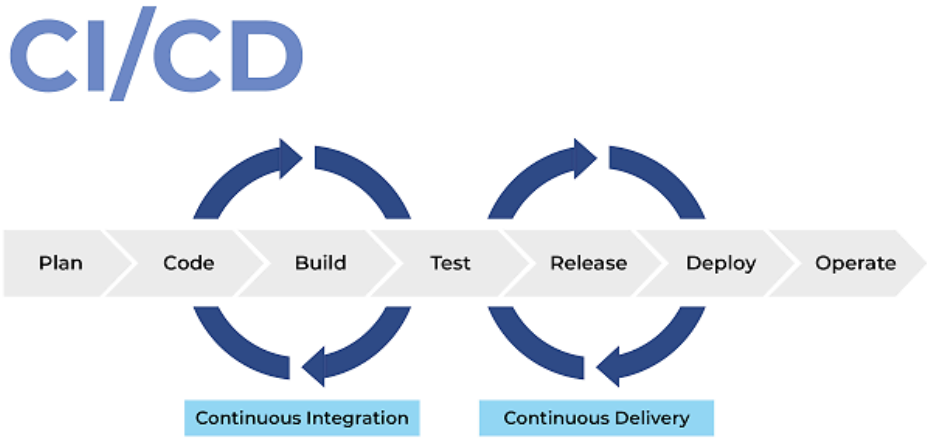
What maturity level is the organisation behind this architecture?

- A: Level 2 - Fully automated
- B: Level 0 - Manual process
- C: Impossible to tell from an architecture diagram
- D: Level 1 - ML pipeline automation

### Answer

**D.** An automated ML pipeline enables rapid experimentation, continuous model delivery and testing, and modularized, re-used code → no training-serving skew.

B is wrong because the training pipeline clearly exists as a deployable artifact. A is wrong because there is no CI/CD that automatically builds, tests, and deploys new pipelines in response to new data or code. C is wrong - these diagrams map directly onto the maturity levels.




From https://www.linkedin.com/pulse/cicd-what-why-important-rob-larter/

At Level 2, what do "continuous integration" and "continuous delivery" apply to?

- A: Only the application code, like classic DevOps
- B: The pipelines: continuous integration of pipelines, continuous delivery of pipelines and models
- C: The code and the models, but not the pipelines
- D: Nothing in particular; CI/CD is not a maturity question

## Answer

**B.** In Level 2, the pipelines are the artifact under CI/CD: new data or new code automatically triggers building, testing, and deployment of new pipeline versions and models.

A is classic DevOps for application code; it is not what distinguishes Level 2. C is close, but it misses that the *pipelines* are what gets integrated and delivered. D is wrong - CI/CD for pipelines is exactly what distinguishes Level 2.

---

Which of the following are true of a Level 0 organisation? Select all that apply.

- A: A person manually deploys the model
- B: Data validation automatically triggers retraining
- C: Model training happens interactively, e.g. in notebooks
- D: Modularized pipeline components are shared between training and serving
- E: No CI/CD

- F: No monitoring

## Answer

**A, C, E, and F.** That is the manual-process level: notebook-driven training, human deployment, no CI, no monitoring.

B is a Level 2 trait - production data automatically triggers retraining. D is a Level 1 trait - it is exactly what removes the training-serving skew.

---

What is "training-serving skew"?

- A: The model trains slower than it serves, so training lags behind production
- B: Serving the model skews the training data towards recent examples
- C: The model behaves differently in production than during training, because the training pipeline and the serving pipeline were built separately
- D: A skew statistic above 1 between the train and the test loss, used to detect overfitting

## Answer

**C.** When the people who train and the people who serve build two different implementations of the same logic, behaviour drifts apart.

A is about speed, not behaviour. B reverses the causality - serving does not touch training data. D invents a statistic; no such metric is part of the term.

---

A model in production keeps degrading and breaking. Which mechanisms would you put in place to catch and fix it? Select all that apply.

- A: Retrain manually whenever a customer complains
- B: Data validation - data schema skews and data value skews
- C: Nothing - models are finished once deployed
- D: Model validation - offline vs. online metrics
- E: Pipeline triggers for automated retraining

## Answer

**B, D, and E.** Validate incoming data, validate the model offline and online, and let the pipeline trigger retraining (next lecture).

A is reactive - by the time a customer complains, the damage is done. C is the Level 0 attitude, and decay compounds over time.



## Data and Model

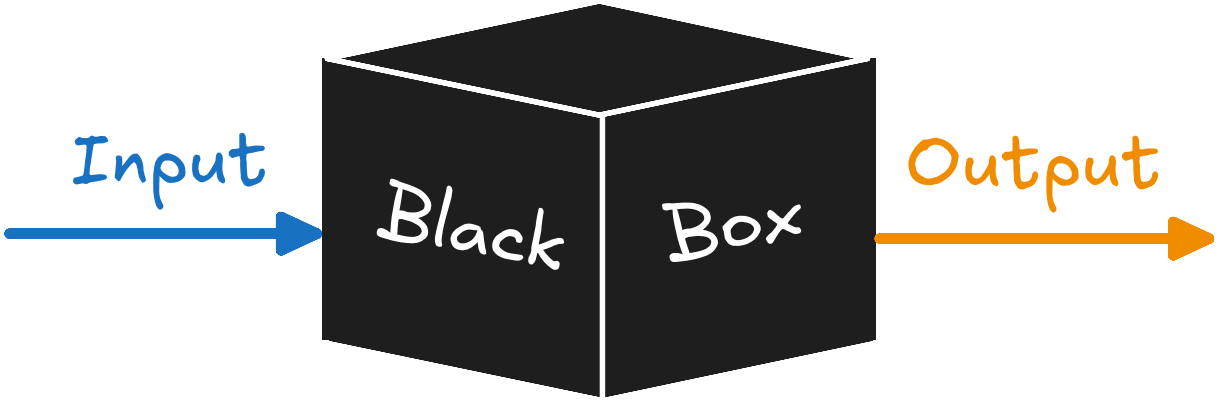
# Data and Model

Recap of the "Data Component" and "Model Component" lectures.

## Discussion

### We keep saying the model is a "black box". Why do we treat it that way - and what follows from it for how we work with models?

- Training maps inputs to outputs without exposing internal logic - we cannot look inside, only observe behaviour
- So we evaluate from the outside: holdout test sets, multiple metrics (error, bias)
- The model must beat a baseline, or it is not worth deploying
- Don't get too attached to one model - feature and model engineering, random search for hyperparameters → robustness is important




## Questions

Of a data scientist's working hours, how much goes to collecting and cleaning data rather than modeling?

- A: Around 5%
- B: Around 60%
- C: Practically 100%
- D: Around 20%

### Answer

**B.** CrowdFlower's survey put ~60% on cleaning (plus ~19% on collecting data), and Anaconda's State of Data Science found ~45% on data loading and preparation.

A and D are far too optimistic - the data work dominates. C overstates it: there is still modeling, evaluation, and communication left.

→ Garbage in, garbage out - the data component deserves the attention



---

What is the typical order of steps in the data component?

- A: Preparation → extraction → validation
- B: Validation → training → extraction
- C: Extraction → validation → preparation
- D: There are no steps; data is data

## Answer

**C.** First you get the data (extraction), then you understand and check it (validation), then you clean and transform it (preparation).

A and B scramble the order - you cannot validate data you have not extracted, and preparation before validation wastes effort on data that may be unusable. D ignores that the pipeline has a deliberate structure.

---

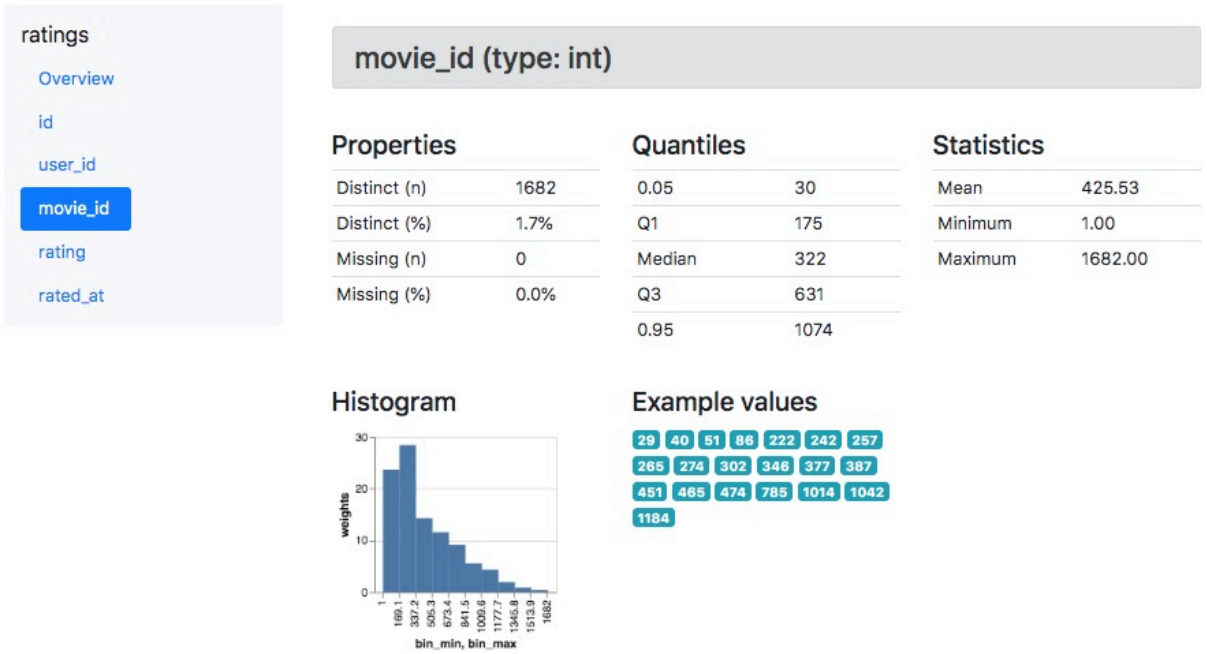
What does data exploration/validation look at? Select all that apply.

- A: Missing values
- B: The model's hyperparameters
- C: Mean and standard deviation
- D: The number of GPUs
- E: Data types

## Answer

**A, C, and E** - the basic shape of the data.

B belongs to model training, not the data. D is a hardware question.




---

Data validation is not just for training - what else is it useful for?

- A: Generating new features automatically from raw data
- B: Making the dataset smaller so training runs faster
- C: Catching problems in incoming data before they poison the model
- D: Ensuring compliance by itself, without further policies

## Answer

**C.** Validation rules double as monitors: when incoming serving or training data breaks the expected schema, you can alert on it.

A, B, and D are not what validation does - feature generation is a preparation step, size reduction is not the goal, and compliance needs policies and processes beyond data checks.

---

What does the "silver" layer refer to?

- A: The most expensive data, e.g. data purchased from a vendor
- B: A backup of the raw data, kept in case the pipeline breaks
- C: Second prize in a Kaggle competition, after the gold layer

- D: The cleaned, transformed data produced after preparation and wrangling

## Answer

**D.** After extraction and validation, preparation turns raw data into the cleaned "silver" layer that models actually train on.

A, B, and C play on the name - the term describes a quality/processing stage, not price, backups, or medals.

---

What must be true before a model is operationalized?

- A: It has been deployed to production
- B: It has the largest parameter count of all candidates
- C: It passes unit/integration tests and is validated to be better than the baseline on a holdout set
- D: It was trained on all available data to make sure it is as up to date as possible.

## Answer

**C.** Unit/integration tests check the model specification, and evaluation on a holdout set must beat the baseline.

A is circular - deployment is what comes *after* operationalization is justified. B rewards size over quality and may be difficult to explain to stakeholders. D leaks the test set: the holdout must stay untouched for the evaluation to mean anything.

---

Why is model packaging tricky? Select all that apply.

- A: Not all serving tools support all formats
- B: Models cannot be saved at all, only retrained
- C: Different frameworks save models in different formats (Pickle, ONNX, TorchScript, PMML, ...)
- D: Formats differ in compression and human-readability

## Answer

**A, C, and D.** The zoo of formats (Pickle, ONNX, TorchScript, PMML, HDF, ...) is exactly why packaging and testing matters.

B is flatly wrong - every framework has a serialization format; the difficulty is compatibility, not existence.

---

Which statement about ONNX is true?

- A: It only works with PyTorch and the PyTorch serving stack
- B: It is a proprietary Apple format used for Core ML apps
- C: It stores the model as human-readable XML
- D: It is an open format supported across many frameworks and runtimes (TF, Core ML, ML.NET, ...)

### Answer

**D.** ONNX is an open format backed by the LF AI & Data Foundation, with support across TensorFlow, Core ML, ML.NET, and more.

A is wrong - it was designed precisely to work across frameworks. C describes PMML (XML), not ONNX, which is a compressed binary format. B confuses it with Apple's .mlmodel.

## Code and Deployment

# Code and Deployment

Recap of the "Code Component" and "Deployment Strategies" lectures.

---

## Discussion

### Think of a machine learning model you have met in real life - as a user, at work, anywhere. What did it do, and how do you think it was served?

Examples that usually come up, and what they reveal:

- Spam filters, credit scoring, churn campaigns → often **batch**
- Recommendations, search ranking, autocomplete → usually **on demand**
- Phone keyboards, face unlock → on-device serving
- Serving to humans or machines? Sanity checks? Fallback? Interpretation outside the model?

---

## Questions

Your model is served from a serverless function. What happens on a cold start?

- A: The first prediction returns instantly, same as any other prediction

- B: The function refuses to start until a container is provisioned manually
- C: The runtime and model must boot and load first, typically taking hundreds of milliseconds up to several seconds
- D: Nothing - serverless functions are always warm once created

## Answer

**C.** With no running instance, the first call pays for booting the runtime and loading the model - hundreds of milliseconds to seconds depending on the model size.

A and D ignore that "no running instance" is the whole point of serverless. B is wrong - provisioning is automatic, it is just *slow* the first time.

→ The trade-off: serverless does not require a running instance, but costs more per inference

---

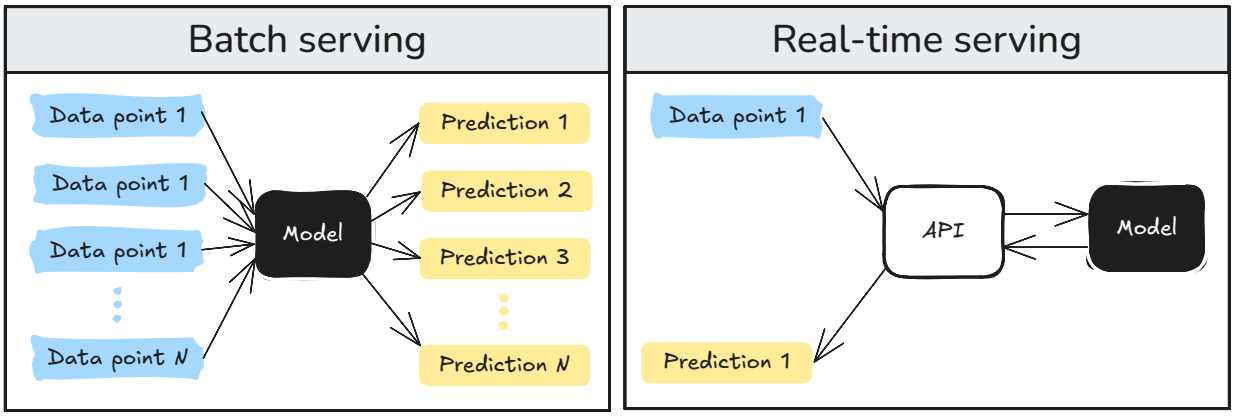
What are the two modes of model serving?

- A: In Docker and in Kubernetes
- B: In batch and on demand
- C: Online and offline learning
- D: Training and testing

## Answer

**B.** Predictions are either computed in bulk ahead of time (batch) or computed when requested (on demand/online/real-time).

A lists deployment technologies, not serving modes. C is about *learning* modes, not serving. D are model development phases.




---

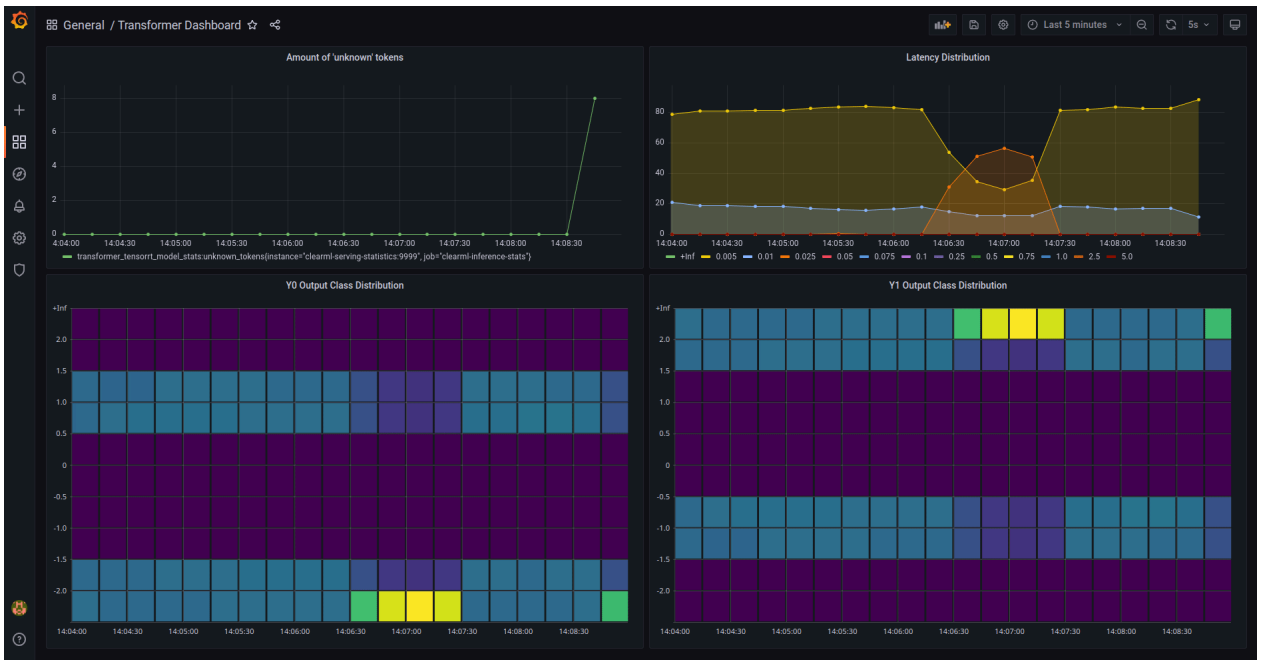
Why log predictions and monitor a model in production? Select all that apply.

- A: Performance deteriorates over time
- B: To increase the parameter count
- C: Data changes
- D: Because the notebook told you to
- E: Abuse/adversarial inputs

## Answer

**A, C, and E.** Models decay, the world moves under them, and people probe them - logging predictions is how you see any of it happening.

B is meaningless - logging does not change the model. D is not a reason.




---

What goes into model maintenance decisions? Select all that apply.

- A: Newer frameworks
- B: Model freshness
- C: Personal preference
- D: Retraining time
- E: Deployment cost

## Answer

**B, D, and E.** How stale the model is, how long a retrain takes, and what a redeployment costs together determine when and how often to retrain.

C may happen in practice, but it is not a criterion to teach. If you want to be structured about it, you could run A/B tests and check if it improves your primary metric (such as clickthrough rate or like-ratio).

A is not a maintenance decision.

---

Docker containers vs. serverless functions for serving - which statement is true?

- A: Serverless is always cheaper and faster than containers at any scale
- B: Serverless does not require a running instance, but is more expensive per inference; containers keep a lightweight instance running
- C: Containers cannot serve models; they are only for training
- D: Serverless functions require you to manage your own servers

## Answer

**B.** That is the trade-off: serverless scales to zero and you pay per inference, containers keep the model warm with predictable latency.

A is wrong in both directions - per inference, serverless is the more expensive option. C is wrong - containers are the standard way to serve models. D contradicts the definition of serverless.

---

What pattern does the course project follow?

- A: Real-time training, offline serving
- B: Batch training, batch serving of a single model
- C: No serving; the model is only evaluated in a notebook
- D: Offline training, on-demand serving through a REST API

## Answer

**D.** The model is trained offline in the pipeline, and served on demand through a REST API.

A flips both - training is not real-time and serving is not offline. B describes a batch-only setup without an API. C describes Level 0, and you are building a pipeline and serving.

---

Given the exercises - Cookiecutter project structure, a local pipeline, CI/CD, and cloud storage - which maturity level is the course project itself?

- A: Level 2 - Fully automated
- B: Level 0 - No MLOps/manual process

- C: Level 1 - Semi-automated data/model/code pipelines

### Answer

**C.** The pipeline and the CI/CD for the code are automated, but a human decides when a new model is worth deploying.

A is wrong - there is no pipeline-triggered retraining; that jump comes in the next lessons. B is wrong - there is an automated pipeline, not a manual notebook-only process.

## Exercise Intro

# Background

Whenever you start a coding project, there's always the question of how you will structure the code.

Where is the main entry point into the code? What util functions do you need to add? Which components does your project require?

There is obviously not *one* correct way of doing it, and it very much depends on what kind of project you're building. A good starting point for that is *Cookiecutter*, which is a tool that can help you set up various projects using whichever template that suits your purpose.



For these exercises, we will use a specific template for Data Science, called Cookiecutter Data Science. This is not by any means the only way to structure a data science project, but the template has well-reasoned arguments as to why.

## Learning Goals

> ***Learning goals*** *By the end of the exercises, we expect you to be able to do the following:*
>
> - *Start a data science project with version control that follows the CCDS format*
> - *Explain the motivation for the structure of a data science project*
> - *Map the typical MLOps steps (data, models, code) onto a concrete project structure*
> - *Run an ML pipeline locally from end to end as a set of composable steps*

> - *Set up a CI/CD workflow that runs your pipeline and produces the model artifact on every change*
> - *Use S3-compatible object storage for data and model artifacts, with credentials kept out of version control*

## Exercises

The following exercises take you from an empty directory to a working MLOps project:

- Exercise 1: Cookiecutter Setup - Scaffold the repo and push it to GitHub
- Exercise 2: Project Structure - Map the MLOps steps from the lecture onto the CCDS structure
- Exercise 3: Local Pipeline - Run the whole ML pipeline locally with a single command
- Exercise 4: CI/CD - Run the pipeline and produce the model artifact automatically with GitHub Actions
- Exercise 5 (extra): Cloud Storage - Store data and model artifacts in S3-compatible object storage

---

# Exercise 0: Setup

Make sure Cookiecutter Data Science is installed: `ccds --version` Does this command run and show a version number above 2.something? Great!

## Cookiecutter Setup

# Exercise 1: Cookiecutter Setup

Make sure Cookiecutter Data Science is installed: `pipx install cookiecutter-data-science`

Try running it with: `ccds --version`

Does this command run and show a version number above 2.something? Great!

### Exercise 1.1: Start your DS project

Let's start a new project using Cookiecutter, inspired by CCDS's documentation on their homepage:

1. **Initialise CCDS in directory**

   In the terminal, run `ccds`

*Discuss what each option does*

2. ### Initialise git in the repo (if relevant)

   In the terminal, run `git init`

3. ### Start tracking new files with git

   ```
   git add .

   git commit -m "Initial CCDS commit"
   ```

   *What does `git add .` do?*
   *Also: what's in `.gitignore`?*

## Exercise 1.2: Push to GitHub

The pipeline you will build in the coming exercises ends up being tested in CI/CD, so the code needs a home on github.com. If you don't want to use your normal github.com user for this course, feel free to create a dummy one.

1. ### Create a new repo on github.com

   Create an empty repository (no README, no .gitignore, no license - you already have all of those).

2. ### Push your code to this new remote (*hint: these lessons covered it*)

   ```
   git remote add origin git@github.com:<username>/<new_repo>.git

   git push
   ```

   *We are actually missing an argument to `git push`. Can you find out what?*

   #### Hint:

   ```
   git push --set-upstream origin main
   ```

3. ### Work on a branch

   Good developer practice is to not work directly on main since that is reserved for production code. Instead create a new branch for the coming exercises:

   ```
   git checkout -b w04-project-setup
   ```

   *Why is it a good idea to keep main "green" (i.e. always working)?*

From here on, work on feature branches and merge them to main via pull requests once the exercises are done - that is the same workflow you will use in any real project.

# Exercise 2: Inspect Project Structure

## Exercise 2.1: The scaffold

Inspect the structure of the resulting repo. It may look something like this:

- `LICENSE` — Open-source license if one is chosen
- `Makefile` — Makefile with convenience commands like `make data` or `make train`
- `README.md` — The top-level README for developers using this project.
- `data/` — Directory for storing data locally
    - `external/` — Data from third party sources.
    - `interim/` — Intermediate data that has been transformed.
    - `processed/` — The final, canonical data sets for modeling.
    - `raw/` — The original, immutable data dump.
- `docs/` — A default mkdocs project; see www.mkdocs.org for details
- `models/` — Trained and serialized models, model predictions, or model summaries
- `notebooks/` — Jupyter notebooks. Naming convention is a number (for ordering), the creator's initials, and a short `-` delimited description, e.g. `1.0-jqp-initial-data-exploration`.
- `pyproject.toml` — Project configuration file with package metadata for `{{ cookiecutter.module_name }}` and configuration for tools like black
- `references/` — Data dictionaries, manuals, and all other explanatory materials.
- `reports/` — Generated analysis as HTML, PDF, LaTeX, etc.
    - `figures/` — Generated graphics and figures to be used in reporting
- `requirements.txt` — The requirements file for reproducing the analysis environment, e.g. generated with `pip freeze > requirements.txt`
- `setup.cfg` — Configuration file for flake8
- `{{ cookiecutter.module_name }}/` — Source code for use in this project.
    - `__init__.py` — Makes `{{ cookiecutter.module_name }}` a Python module
    - `config.py` — Store useful variables and configuration
    - `dataset.py` — Scripts to download or generate data
    - `features.py` — Code to create features for modeling
    - `modeling/`
        - `__init__.py`

        - `predict.py` — Code to run model inference with trained models
        - `train.py` — Code to train models
    - `plots.py` — Code to create visualizations

How does this structure make sense to you?

- Do you understand where the ML code goes?
- What does each of the pre-generated files do?
- Any terms that you don't know what they mean?
- Other confusing things?

## Exercise 2.2: Where do the MLOps steps live?

In the lecture we walked through the typical steps of an ML pipeline that consisted of a **data component**, a **model component** and a **code component**. Now it's time to put this into code.

1. Where does each step live in this CCDS structure?
    - Data extraction/ingestion
    - Data validation
    - Data preparation
    - Model training
    - Model evaluation
    - Model packaging
    - Inference/serving
2. Copy the files in https://github.com/Jeppe-T-K/ccds-dsip-exercise-repo/tree/main/pipeline_steps into your project and validate that you can run the steps.
3. Where would you put the other code parts that we've covered so far?
    - Dockerfile(s)
    - Tests
    - Workflows

You now have a mental map of the project. In the next exercise, you will try to build a pipeline for this using a single command.

## Local Pipeline

# Exercise 3: Run the Pipeline Locally

In Exercise 2 you mapped the MLOps steps onto your project structure. In this exercise you will run

that pipeline the MLOps level 1 way: with a single command.

## Exercise 3.1: Get set up

Carry your project forward from the previous exercises. Work in the repo you scaffolded in Exercise 1 and added different scripts to in Exercise 2.

### If you haven't set up a project yet, you can use this one instead

It is a CCDS scaffold with example implementations of each pipeline stage in `pipeline_steps/` filled into `src/`:

```
git clone https://github.com/Jeppe-T-K/ccds-dsip-exercise-repo.git my-ds-project
cd my-ds-project
```

Either way you should recognize the CCDS layout from Exercise 2:

- `src/dataset.py` fetches or generates the raw data
- `src/features.py` prepares it
- `src/modeling/train.py` trains a model
- `src/modeling/predict.py` runs batch inference

### Questions

### Can you run each component by itself?

- Ideally yes. We aim for modularity, so each step should be possible to run without the other. However, the training step might require that some data has been fetched/processed.

### How do you handle python packages?

- Depending on what you installed:
  - uv: `uv add ...`
  - requirements.txt: add e.g. `sklearn==1.3` in file
  - conda: `conda install <package>` (often via `-c conda-forge`)
  - etc...

## Exercise 3.2: Run the pipeline with the Makefile

For this exercise, we keep it simple and make use of the Makefile CCDS uses.

> *For the project we expect you to use Go/Dagger for the pipelines.*

In the Makefile you can define how to run each step. CCDS already has a few pre-filled ones, and so you can glimpse the pattern directly:

```
.PHONY: requirements  # runs as recipe, doesn't look for file
requirements:         # make <command>, e.g. `make requirements`
    uv sync           # Command(s) to run. Can be multiline
```

You can modify the `Makefile` so each `make <command>` is easy. Create a command for each of the following:

- `data`
- `features`
- `train`
- `predict`
- plus a target that chains them all in order (e.g. `pipeline`)

Run it, then inspect what appeared under `data/` and `models/`. Try running the individual targets too, e.g. `make train` alone. If you cloned the example repo, make sure uv is installed - the Makefile uses it to sync dependencies.

### Questions

### How does this differ from using Dagger for creating the pipeline?

- No specification of Docker image
- No environment isolation: `make` runs commands in your local Python environment, so results depend on what is installed on that machine; Dagger steps run in containers and are reproducible anywhere
- No caching of intermediate results: the targets are phony, so every step re-runs every time; Dagger caches each step's outputs and only re-runs what changed

### What is still missing before you would call this MLOps level 1?

- Automation: the pipeline only runs when you type `make pipeline` - a human is the trigger. Level 1 means the pipeline runs itself, e.g. on a schedule or when new data arrives, without anyone remembering to run it.
- CI: pushing to GitHub triggers nothing. There is no automatic run of the tests, and no way for anyone (including future you) to see that the pipeline still works after a change. That is what you build in Exercise 4.
- (The same goes for where the artifacts live: everything is still only on your machine - the extra Exercise 5 fixes that.)

## CI/CD

# Exercise 4: Automate with GitHub Actions (CI/CD)

Right now, your pipeline runs when *you* remember to run it. In the lecture's maturity levels, that is stage 1 at best. The defining trait of stage 2 is that every change to the code is automatically built and delivered - continuous integration and continuous delivery.

There are two parts to getting there. First, express your pipeline as a **Dagger workflow in Go**, the way you learned in the workflows lesson. Then, let GitHub run that same workflow for you on every push: a workflow file in your repo describes what to do, and GitHub runs it on a fresh virtual machine.

In this exercise you will turn the `make pipeline` chain from Exercise 3 into a Dagger pipeline - and then put it in CI so it produces your model artifact automatically.

## Exercise 4.1: The pipeline as a Dagger workflow

### Exercise 4.1.a: Create the workflow

Carry your project forward from Exercise 3. You will add the pipeline files next to the CCDS structure, so your repo looks like:

```
my-ds-project/
├── go.mod              <- the Dagger pipeline's Go module
├── pipeline.go         <- the Dagger pipeline
├── src/                <- your pipeline steps (dataset, features, train, predict)
├── data/
├── models/
└── Makefile            <- the old way of chaining the steps
```

1. **Set up the Go module**

   You need Go installed, and a running container runtime (Docker Desktop or similar) - Dagger starts its engine through it.

   In the repo root, initialize the module and create the pipeline file, like in the lesson. We put the module in the repo root rather than its own directory, because the pipeline needs to see your whole project:

```
go mod init daggerpipeline
touch pipeline.go
```

2. ## Start from the lesson template

   Start from the skeleton you know from the lesson: a `main` that creates a shared context and calls the pipeline function, and inside it a Dagger client, a container built from a Python image, and a step that runs one of your Python files:

```
package main

import (
    "context"
    "fmt"

    "dagger.io/dagger"
)

func main() {
    ctx := context.Background()

    if err := Pipeline(ctx); err != nil {
        fmt.Println("Error:", err)
        panic(err)
    }
}

func Pipeline(ctx context.Context) error {
    client, err := dagger.Connect(ctx)
    if err != nil {
        return err
    }
    defer client.Close()

    // ...
}
```

   Two differences from the lesson's `hello.py` example:

   - Your code and dependencies live in the **repo root**, the same directory you run from - so the host directory is simply `.`.
   - Your steps need your project's dependencies installed, not just a plain Python image.

3. ## Fill in the data step

   Load the project into a Python container, install your dependencies from the lockfile, and run the data step - the same command your `make data` target runs. Then export the container's `data/` back to the host:

```
func Pipeline(ctx context.Context) error {
    client, err := dagger.Connect(ctx)
    if err != nil {
        return err
    }
    defer client.Close()

    // Only copy what the pipeline needs into the container.
    // data/ and models/ are not on the list, so the container
    // always starts from a fresh machine - the pipeline must
    // build everything itself.
    project := client.Host().Directory(".", dagger.HostDirectoryOpts{
        Include: []string{
            "src",
            "pyproject.toml",
            "uv.lock",
        },
    })

    // Base container: uv image with your Python version,
    // with dependencies installed from the lockfile
    python := client.Container().
        From("ghcr.io/astral-sh/uv:python3.14-bookworm").
        WithDirectory("/app", project).
        WithWorkdir("/app").
        WithExec([]string{"uv", "sync"})

    // Stage 1: Data
    data := python.WithExec([]string{"uv", "run", "python", "src/dataset.py"})
    if _, err := data.Directory("data").Export(ctx, "data"); err != nil {
        return err
    }

    return nil
}
```

Run `go mod tidy` to fetch the Dagger dependency.

Notes:

- The `uv` commands mirror your Makefile targets: `uv sync` installs exactly the lockfile-pinned dependencies, and `uv run python src/dataset.py` is the same command `make data` runs. Match the image's Python version to the one in your `pyproject.toml`. If your scaffold does not use uv, use a plain Python image (e.g. `python:3.12-bookworm`) and `pip install -e .` instead.
- `Export` copies the container's `data/` directory out to the repo's `data/` - the same `data/` you inspected after `make data` in Exercise 3.

4. ## Add the remaining stages: features, train, predict

The pattern is now the same for every stage: take the container, run the step's command,

export what the step produced. Because each stage starts from the previous stage's container, the steps chain in order - and Dagger caches every command, so a stage whose inputs did not change is not re-run.

```
// Stage 2: Features
features := data.WithExec([]string{"uv", "run", "python", "src/features.py"})
if _, err := features.Directory("data").Export(ctx, "data"); err != nil {
    return err
}

// Stage 3: Train
train := features.WithExec([]string{"uv", "run", "python", "src/modeling/train.py"})
if _, err := train.Directory("models").Export(ctx, "models"); err != nil {
    return err
}

// Stage 4: Predict
predict := train.WithExec([]string{"uv", "run", "python", "src/modeling/predict.py"})
if _, err := predict.Directory("models").Export(ctx, "models"); err != nil {
    return err
}
```

Adapt the commands to whatever your Makefile targets run, and match the directories your project actually writes to (`data/raw/`, `data/processed/`, `models/`, ...). If a step writes to multiple directories, export each of them.

## Exercise 4.1.b: Run the pipeline locally

1. ### Run it

   From the repo root:

   ```
   go run pipeline.go
   ```

   The first run takes a while - Dagger pulls the Python image and installs your dependencies. Then inspect what appeared under `data/` and `models/`, like you did in Exercise 3.

2. ### Feel the caching

   Run `go run pipeline.go` again, immediately. Then make a small change to `src/dataset.py` and run once more.

   - On the second run, nothing re-runs: every command's inputs are unchanged, so Dagger replays the cached layers and only re-exports the artifacts. `make` re-ran everything, every time.

- On the third run, the container image and dependency install are cached, but the data step's input (your source code) changed, so the data step - and everything downstream of it - re-runs.

# Exercise 4.2: Put it in CI/CD

## Exercise 4.2.a: Create the workflow file

### 1. Create the workflow file

GitHub Actions gives you CI/CD for free: a workflow file in your repo describes what to do on every push or pull request, and GitHub runs it on a fresh virtual machine. Since your pipeline is now a Dagger workflow, you embed it the way you learned in the workflows lesson.

Create `.github/workflows/ci.yml`:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  pipeline:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v7.0.1

      - name: Set up Go
        uses: actions/setup-go@v7.0.0
        with:
          go-version: 1.26.1
          cache: false

      - name: Run Dagger pipeline
        uses: dagger/dagger-for-github@v8.4.1
        with:
          verb: run
          args: go run pipeline.go
          version: "0.21.11"
```

Commit it, push, and watch the run under the **Actions** tab on GitHub.

The workflow does not restate your pipeline - it runs the *same* `pipeline.go` you ran locally. One definition, two environments. That is the payoff for expressing the pipeline as code in 4.1.

### Exercise 4.2.b: Watch it run

1. **Read the run**

   Remember: the runner is a fresh machine - nothing in `data/` or `models/` exists until your pipeline creates it. You already designed for that in 4.1.a, by not copying them into the container.

   The exports land in the repo root on the runner, and both are deleted with the machine when the job ends. The runner needs Go and the Dagger engine - but no Python, no uv, no virtualenv: your pipeline brings its own in the container.

### Exercise 4.2.c: Publish the artifact

Continuous *delivery* means the pipeline's output is produced automatically, not just run.

1. **Upload the model**

   Add a step that uploads what the pipeline produced with `actions/upload-artifact`:

   ```
   - name: Upload model artifact
     uses: actions/upload-artifact@v4
     with:
       name: model
       path: models/
   ```

   Once the run finishes, download the artifact from the run page. This is exactly how trained models are handed from a training workflow to whatever comes next: a validator, a deployment job, or a colleague.

Back to Introduction

## Cloud Storage

# Exercise 5 (extra): Data and Models in S3-Compatible Object Storage

Your pipeline now runs locally, but everything it produces lives on your laptop. If a teammate (or a CI machine) clones your repo, `data/raw/` and `models/` are empty. Git is for code - data and model artifacts belong somewhere else.

In practice, that "somewhere else" is almost always an object store speaking the **S3 API**. The S3 protocol started as an AWS service, but is now the de-facto standard supported by everyone: DigitalOcean Spaces, Google Cloud Storage (S3-compatible API), Azure (via gateways), and self-hosted solutions like MinIO. Learning the S3 API once means you can talk to all of them.

In this exercise you will make your pipeline pull its raw data from and push its model artifact to an S3-compatible bucket, first locally with MinIO, then (optionally) against a real cloud provider.

## Exercise 5.1: A local object store with MinIO

MinIO is an S3-compatible object store you can run locally - perfect for development, and it means you don't need cloud credentials to do this exercise. You already know the tool for the job from the containers lesson:

### 1. Start MinIO in Docker

```
docker run -d --name minio \
  -p 9000:9000 -p 9001:9001 \
  -e MINIO_ROOT_USER=minioadmin \
  -e MINIO_ROOT_PASSWORD=minioadmin \
  minio/minio server /data --console-address ":9001"
```

Open http://localhost:9001 and log in with `minioadmin` / `minioadmin`. The console looks different from AWS, but every button here has an S3-API equivalent.

### 2. Create a bucket for your project

Create a bucket, e.g. `my-ds-project`, either via the console or with `mc`.

```
Given the data stages from the lecture, what "folders" (S3 uses key prefixes, not real f
```

## Exercise 5.2: Talk S3 from Python with boto3

`boto3` is the standard Python client for the S3 API - it works against AWS, Spaces, and MinIO alike, you just point it at a different `endpoint_url`.

### 1. Install and configure credentials outside version control

Add `boto3` to your project dependencies. Then create a `.env` file (check that `.gitignore` ignores it - CCDS does this for you):

```
AWS_ACCESS_KEY_ID=minioadmin
AWS_SECRET_ACCESS_KEY=minioadmin
S3_ENDPOINT_URL=http://localhost:9000
S3_BUCKET=my-ds-project
```

*Why must credentials never be committed? What could someone do with a leaked key to a re*

2. ### Write a small storage helper

   In your package, add helpers for upload/download. The interesting part is that the client is configured from the environment, so the same code works against MinIO *and* a real cloud:

   ```
   import os
   import boto3

   def s3_client():
       return boto3.client(
           "s3",
           endpoint_url=os.environ["S3_ENDPOINT_URL"],
       )
   ```

   Try listing the objects in your bucket from Python (`client.list_objects_v2`) to verify the connection.

3. ### Upload something

   Upload the raw data from Exercise 3 to `s3://<bucket>/data/raw/...` and verify you can see it in the MinIO console (or with `mc`).

   *What happens if you upload the same key twice? What does that tell you about using the S*

## Exercise 5.3: Put the pipeline on storage

Now make the pipeline actually use the bucket:

1. ### Ingestion pulls from S3

   Change the data ingestion step to download the raw data from the bucket to `data/raw/` if it isn't there already (rather than generating it locally). Local files are now a *cache* of the bucket, not the source of truth.

2. ### Training publishes to S3

   After training and evaluation, upload the serialized model and `metrics.json` to the bucket under a key that includes some notion of version or timestamp, e.g. `models/<version>/model.pkl`.

3. ### Prove it is reproducible

   Delete `data/` and `models/` locally and re-run your pipeline (`make pipeline` or `go run pipeline.go`). Then simulate a fresh teammate: clone your repo into a new directory and run the pipeline there.

```
What still breaks on a fresh clone that didn't break on your machine? (hint: where do th
```

## Exercise 5.4: Discuss

```
Why do you think basically every MLOps platform (including the tools from the data versioni

What are the trade-offs of storing artifacts in a bucket vs. committing them to git vs. att

Your MinIO data disappears when you delete the container. What would you use instead for so
```

## Optional: try a real provider

If you have a cloud account (DigitalOcean gives free credit to students), create a Spaces bucket (or an S3 bucket on AWS), generate an access key, and point your `.env` at it. Nothing in your code should need to change - that is the point of the S3 API being a standard. Note that Spaces uses `https://<region>.digitaloceanspaces.com` as the endpoint URL.